

Text2BIM：使用基於大型語言模型的多智慧體生成建築模型

框架

Changyu Du¹、Sebastian Esser²、Stavros Nousias³ 和 André Borrmann⁴

¹博士生，計算建模與仿真系主任，慕尼黑工業大學，慕尼黑

80333，德國。電子郵件：changyu.du@tum.de

²慕尼黑工業大學博士後研究員、計算建模與仿真系主任

德國慕尼黑 80333。電子郵件：sebastian.esser@tum.de

³慕尼黑工業大學博士後研究員、計算建模與仿真系主任

德國慕尼黑 80333。電子郵件：stavros.nousias@tum.de

⁴慕尼黑工業大學教授、計算建模與仿真系主任，慕尼黑 80333，

德國。電子郵件：andre.borrmann@tum.de

抽象

傳統的 BIM 創作過程通常需要設計師掌握複雜而乏味的建模命令，以便在 BIM 創作工具中實現他們的設計意圖。這種額外的認知負擔使設計過程複雜化，並阻礙了 BIM 和基於模型的設計在 AEC（建築、工程和建築）行業。為了更直觀地表達設計意圖，我們支援 pose Text2BIM，一個基於 LLM 的多智慧體框架，可以從自然語言生成 3D 建築模型指示。該框架編排多個 LLM 代理進行協作和推理，從而轉換文字使用者輸入到命令式代碼中，該代碼調用 BIM 創作工具的 API，從而生成可編輯的 BIM 模型內部佈局、外部包絡和語義資訊直接在軟體中。此外，基於規則的模型檢查器被引入代理工作流程中，利用預定義的領域知識來指導 LLM 代理解決生成的模型中的問題並反覆運算提高模型品質。廣泛的實驗進行了比較和分析所提出的框架下三種不同 LLM 的性能。這評估結果表明，我們的方法可以有效地產生高品質、結構合理的建築與使用者輸入指定的抽象概念一致的模型。最後，互動式軟體原型旨在將框架集成到 BIM 創作軟體 Vectorworks 中，展示了通過聊天建模。

1 介紹

在過去的幾十年裡，各種數位表示和工作流程不斷出現，以表示

建築環境。建築資訊模型（BIM）的概念包括一種整體方法來反映具有幾何和語義資訊的建築資產，可在建築物的整個生命週期中使用並在不同的專案利益相關者之間以專門的表示形式共用（Borrmann 等人，2018 年）。現代 BIM 創作軟體包含跨多個學科的設計要求。這種綜合方法導致了軟體中功能和工具的激增，使用戶介面變得越來越複雜。設計師經常面臨陡峭的學習曲線，需要廣泛的培訓才能將設計意圖轉化為複雜的命令流程在軟體中創建建築模型（Du 等人，2024a）。

近年來，生成式人工智慧（AI）在建築設計中的應用緩解了這種情況額外的認知負荷，增強設計過程的創造潛力和效率。當前研究和工業應用主要集中在生成 2D 圖像或簡單的 3D 體積（Li et al. 2024a），利用用於創建 2D 的變分自動編碼器（VAE）、生成對抗網路（GAN）和擴散模型建築平面圖（Luo 和 Huang 2022;Shabani 等人，2023 年;Wang et al. 2021）、建築效果圖（Stigsen 等人，2023 年;Chen 等人，2023 年;Graphisoft 2024）、建築立面設計（Sun 等人，2022 年;Zhang et al. 2022），或初步的 3D 概念形式（Zhuang 等人，2023 年;Pouliou 等人，2023 年;Tono 和 Fischer 2022）。最近的研究建議使用大型語言模型（LLM）自動生成牆壁細節（Jang 等人，2024 年），但更多複雜的 3D 建築模型生成尚未開發。

在其他領域，例如遊戲開發和虛擬實境，像 DreamFusion 這樣的高級 3D 生成模型（Poole 等人，2022 年）和 Magic3D（Lin 等人，2023 年）可以直接從文本描述，允許設計人員以自然語言表達設計意圖，而無需繁瑣的建模命令。但是，這些文本到 3D 方法的輸出通常基於體素、點雲、網格或隱式像神經輻射場（NeRF）（Mildenhall et al. 2021）這樣的表示形式，它僅包含外表面，無法對 3D 物件的可能內部內容進行建模，也不包含任何語義資訊。

這些純幾何 3D 形狀和原生 BIM 模型之間的差異使得整合變得具有挑戰性它們融入到 BIM 的建築設計工作流程中。設計人員無法直接修改和編輯生成的內容在 BIM 創作軟體中，由於缺乏語義資訊，這些模型也很難應用於下游建築類比、分析和維護任務。

1.1 新穎性和貢獻

為了彌合這些差距，我們提出了 Text2BIM，它將自然語言描述轉換為 3D 建築模型帶有外部包絡、內部佈局和語義資訊。通過將構建模型表示為命令代碼文本調用 BIM 創作軟體的應用程式介面（API），我們啟用多個大型語言模型（LLM）代理進行協作並自主生成可執行代碼，最終生成

能夠直接在軟體中進一步編輯的原生 BIM 模型。擬議的框架利用了特定的提示指導 LLM 生成架構方面的合理結果，並自動評估生成的根據特定領域的建築規則建立 BIM 模型。它允許 LLM 通過以下方式反覆運算提高模型品質多個反饋迴圈，整合來自基於規則的系統的領域知識。各種複雜的測試用例旨在全面評估框架的生成能力和結果品質。我們還實施了互動式軟體原型，以將建議的框架集成到 BIM 創作工具中 Vectorworks 軟體，展示了在設計過程中通過聊天建模的新可能性。

本文的範圍僅限於在早期設計階段生成常規建築模型。生成的模型僅包括基本的建築構件，例如內牆和外牆、樓板、屋頂、門和窗。以及具有代表性的語義資訊，如 Stories、Spaces 和 Material Definition。我們的目標是生成合理佈局的 3D 建築物，具有來自自然語言描述的一定級別模型品質，提供設計師，以進一步優化 BIM 創作軟體中的設計。這種方法旨在部分將設計師從繁瑣重複的建模命令中解放出來，更直觀地表達設計意圖。儘管如此，用戶可以隨時在 BIM 創作工具中繼續修改生成的模型，確保自動化和工程自主性達到平衡水準。

2 相關工作

2.13D 建築設計中的生成式 AI

生成式 AI 在 3D 建築設計領域的應用正逐漸成為研究熱點。關鍵在於為現有設計數據、經驗知識和物理原理，然後訓練相應的演算法以智慧生成新設計（Liao et al. 2024）。de Miguel Rodríguez et al.（2020）使用連接向量來表示不同的 3D 網格狀建築幾何形狀。由使用這些數據訓練變分自動編碼器（VAE），通過重構可以生成更多的構建形狀學習分佈中的插值位置。Vitruvio（Tono 和 Fischer 2022）使用 occupancy 字段來通過為 3D 空間中的每個點分配二進位值來描述建築物形狀，以指示該點是否為被物件佔用。他們採用改進的佔用網路（Mescheder 等人，2019年）來學習這種表示，支援從單個透視圖草圖重建 3D 可列印的建築網格。

其他研究人員正在使用 GAN 為早期設計階段生成概念 3D 建築。普利烏等人（2023年）提議使用 CPCGAN（Yang 等人，2021 年）生成建築物幾何形狀的點雲表示基於特定的網站規則。Ennemoser 和 Mayrhofer-Hufnagl（2023）將 3D 體素解碼為 2D 圖像以進行訓練 a DCGAN（Radford et al. 2015），然後使用有符號距離場（SDF）（Oleynikova et al. 2016）進行轉換生成的圖像返回體素。儘管他們的方法生成的體素模型可以部分重建建築物的內部空間，生成的結果仍然存在幾何不一致和

不準確，使它們與實際的建築模型相去甚遠。

總體而言，目前利用基於純幾何表示的 3D 生成演算法的研究仍然有限。生成概念建築形式，並且還無法使用高開發級別（LOD），同時表現出連貫的外部 and 內部幾何體。此外，結果由這些純數據驅動的方法生成的數據很難使用基於文本的架構規則進行約束。更好的數據需要探索表示方法。

快速發展的生成式 AI 技術，例如擴散模型和大型語言模型（LLM），在建築設計領域顯示出巨大的潛力。擴散模型的應用仍然存在。主要關注基於 2D 圖像的任務，例如從文本生成建築渲染（Li 等人，2024b）並取代 GAN 以產生更強大的結構設計（He et al. 2023）。另一方面，應用 LLM 主要集中在使用自然語言從 BIM 模型中檢索數據（Zheng 和 Fischer 2023）和增強 BIM 創作軟體中的人機交互（Du 等人，2024b; Fernandes 等人，2024 年）。然而使用 LLM 生成 3D 建築模型的概念尚未得到探索。主要挑戰之一是將 3D 模型表示為 LLM 可以使用的一維文本數據（Liao 等人，2024 年）。此文本表示必須簡明扼要地捕獲模型的特徵，避免由過於詳細的粒度典型引起的冗長的常規模型序列化檔。最近的一項研究（Jang 等人，2024 年）提議將 BIM 模型轉換為 XML 格式，然後使用 LLM 處理此結構化文字以添加牆詳細資訊。最後，轉換修改後的 XML 返回到 BIM 模型中，實現自動牆面詳圖設計。與他們的方法不同，我們建議表示 BIM model 作為命令式代碼。通過在 BIM 創作工具中構建和調用高級建模 API，我們的目標是使用盡可能最小和最靈活的文本格式來表達模型的幾何和語義特徵。同時還可以最大限度地利用 LLM 強大的代碼生成功能。

2.2 基於 LLM 的代理

基於大型語言模型的代理是指利用高級語言模型來執行涉及感知、決策和行動的任務（Wang 等人，2024 年）。這些系統通常配備 LLM 具有與外部環境交互的工具，以及用於保留思維過程的記憶體模組，觀察結果和作記錄。基於 LLM 的代理將 LLM 用作“大腦”，利用其強大的上下文學習（Dong et al. 2024）綜合來自各種來源的資訊並部署適當工具的能力。通過語言推理針對不同的場景，從而使系統能夠執行任務。就像人類一樣（Du 等人，2024b）。代理可以單獨作，也可以在他們協作的多代理系統中作，溝通，並專注於不同的角色，以使用集體智慧解決更多動態和複雜的問題（Guo 等人，2024 年）。

基於 LLM 的多智慧體系統在軟體開發（Hong et al. 2023）、遊戲（Xu

et al. 2024) 和社會類比 (Park et al. 2023) 等。然而，它們在 3D 設計中的使用仍處於探索階段。3D-GPT (Sun 等人，2024 年) 框架將 3D 建模任務分解為多個步驟，協調三個步驟 LLM 代理以類似於人工團隊的方式在 Blender 中創建與文本描述匹配的 3D 資產。場景工藝 (胡 et al. 2024) 利用多個 LLM 代理將文本描述轉換為可在 Blender 中執行的 Python 腳本，自動渲染適合在遊戲和電影中使用的 3D 場景。該系統採用場景圖進行類比資產之間的空間關係，並使用視覺語言模型反覆運算優化場景。在架構中，與上述研究類似，Çelen et al. (2024) 使多個 LLM 代理能夠創建場景圖根據使用者指令，然後使用回溯演算法放置傢俱，最終生成室內設計場景。Mehta 等人 (2024 年) 提出了一個互動式框架，允許人類架構師與 LLM 代理使用自然語言指令構建結構。代理可以放置區塊、尋求澄清、並將人類反饋集成到類似 Minecraft 的 3D 模擬環境中。Du et al. (2024b) 整合了 LLM 代理到 BIM 創作軟體中，以通過 Retrieval Augmentation Generation (RAG) 回答軟體使用問題 (Lewis 等人，2020 年)，並根據自然語言指令執行簡單的建模任務。總之，我們領域的現有文獻僅探討了基於 LLM 的代理在簡單建模中的應用上下文。目前缺乏探索多智慧體系統生成複雜 3D BIM 模型。

2.3 基於 BIM 的模型檢查

隨著 BIM 概念在建築資產的設計階段不斷採用，豐富的資訊庫提供了為多種下游應用奠定了複雜的基礎。行業基礎課程 (IFC) (ISO 2024) 數據模型已經建立起來，可以交換由幾何和語義組成的建築資產的數位表示資訊。這些表示是完美定製的，可以自動執行有關合規性的檢查根據各種規則和準則進行設想的設計。這種方法越來越受到過去幾年中該行業的不同利益相關者。機會和 Preidel 和 Borrmann (2018) 描述了相關挑戰。Eastman 等人 (2009 年) 引入了基於 BIM 模型的自動化規範合規性檢查的總體方法。他們劃分整體檢查過程分為四個階段：規則解釋以創建機器可讀的規則，使用 Advanced 構建模型準備分析、規則檢查執行以及詳細缺陷和問題的報告。

模型應遵守的規則的複雜程度可能會有所不同。為了應對這一挑戰，Solihin 和 Eastman (2015) 引入了規則分類系統，該系統包括四個不同的級別。規則分配給 1 類的 API 需要單個或少量的顯式數據可用。此類規則的典型範例是為模型中的每個元素分配的專用屬性的檢查。第 2 類規則的特點是簡單屬性值的派生。此類計算可以包括簡單的算術或三角計算

基於幾何表示或語義資訊的聚合。歸入第 3 類的規則要求擴展的數據結構和處理。這樣的規則需要對語義和幾何進行綜合處理數據和中間計算的評估。此類規則的突出例子可以在該地區找到與消防安全法規相關的評估。這些規則通常涉及對不同的元件、幾何特徵和路徑搜索演算法，用於識別相關的空間和走廊檢查。第 4 類考慮了無法通過規定特徵評估但需要整體評估的規則提取的資訊。在大多數情況下，這些規則考慮了多個目標，這很困難（甚至不可能）以按順序工作流程進行表述。因此，預計軟體應用程式支援使用者提取和識別相關的模型資訊，但最終不在檢查結束時提供簡單的通過/失敗語句跑。

除了對規則的不同複雜性進行研究外，其他研究人員還專注於翻譯人類可讀的指南和法規（Zhang 和 El-Gohary 2017;周 et al. 2022;Fuchs 等人，2022 年）和提出了以機器可讀表示形式制定規則的不同方法（Sydora 和 Stroulia 2020; Häußler 等人，2021 年）。作為最近的發展，開發了資訊傳遞規範（IDS）標準並由 buildingSMART International 維護，支援針對基本屬性檢查的規則規範以統一且獨立於供應商的方式（Tomczak 等人，2022 年）。然而，在目前的發展階段，它僅支援語義資訊，但缺少指定綜合幾何條件的選項。Nuyts 等人。

（2024 年）調查了當前合規性檢查技術的不同方法，並討論了優勢和缺點。作為對已經提到的方法的擴展，他們還考慮了與鏈接相關的技術數據，這些數據早先由（Pauwels et al. 2011）提出。

2.4 總結和確定的研究差距

許多研究已經探索了使用生成式 AI 來創建概念建築的幾何表示。然而，這些進步尚未整合到 BIM 驅動的建築設計領域。基於進行了文獻綜述，我們的方法似乎是第一個利用協作 LLM 代理生成基於自然語言指令的具有相對較高 LOD 的 BIM 模型，確保合規性和一致性通過採用規則驅動的模型檢查。

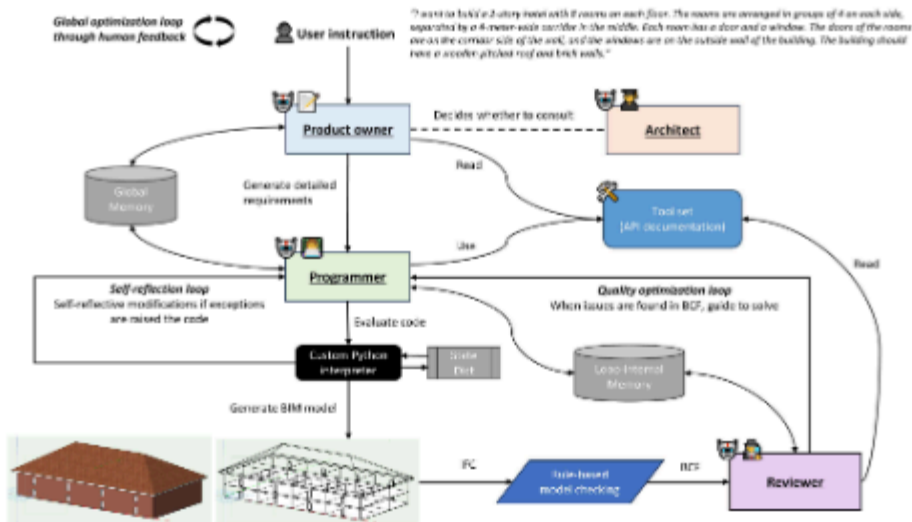


圖 1.擬議的基於 LLM 的多代理框架，帶有示例使用者指令

3 方法論

我們提出了 Text2BIM，這是一個基於 LLM 的多代理框架，其中四個 LLM 代理承擔不同的角色和協作將自然語言指令轉換為命令式代碼，從而在 BIM 中生成建築模型創作軟體。核心思想是使用一系列自定義 API 封裝軟體的底層建模 API 高級工具功能。通過使用提示工程技術來指導 LLM 在生成的代碼，我們可以通過簡潔高效的文本表示來構建原生 BIM 模型。

帶有示例使用者輸入的整個框架如圖 1 所示。為了實現上述核心概念，我們使用四個基於 LLM 的代理，這些代理具有專門的任務和技能，它們通過文字相互交互：

- 產品負責人：改進和增強使用者說明並生成詳細的需求文檔。
- 建築師：根據建築知識制定文本建築計劃。
- 程式師：分析需求並編寫用於建模的代碼。
- Reviewer：提供代碼優化建議以解決模型中發現的問題。

由於使用者輸入通常簡短且開放的性質，我們首先設計了一個 LLM 代理，作為 Product Owner 來擴展和完善用戶說明。這可確保指令包含足夠的資訊，以便指導下游 Programmer 代理在其代碼中調用合適的工具函數。Product Owner 代理的原始說明的闡述和詳解引用了多個來源。首先，它從

自訂工具集，其中包含建模函數和相應的文本描述。這允許代理要瞭解調用相應函數所需的輸入參數是否在使用者的指示或是否需要其他資訊。其次，它利用了 Architect 代理的知識。當產品負責人認為需要更多的架構背景或建築規劃時，尤其是在建築要求複雜，可以選擇諮詢建築師。Architect 代理旨在根據特定建築，以結構化文本格式生成帶有座標和尺寸的建築平面圖規則，並結合 Product Owner 轉發的使用者指令。

最初的使用者說明，經過產品負責人的增強后，成為一個詳細的需求文檔，以指導 Programmer 代理組合和調用工具集中的相應函數構建使用者期望的建築模型。生成的代碼由自定義 Python 解釋器使用語法檢查。如果在代碼執行過程中引發異常，則會自動提示 Programmer 自我反射並反覆運算改進代碼，直到錯誤得到解決。

成功生成的建築模型將自動匯出為基於 IFC 的表示，然後發送到下游模型檢查器進行自動質量評估。我們定製了一系列特定領域的規則，從各個角度全面評估模型品質，包括幾何分析、碰撞檢測、信息驗證等校驗結果最終將在 BIM Collaboration 中導出格式（BCF），包含在模型中發現的問題的描述以及關聯的建築構件指南。

在此階段，引入了 Reviewer 代理來解釋 BCF 檔並提供優化型。Reviewer 旨在通過讀取 BCF 檔。然後，它通過將此資訊與程式師之前生成的信息相結合來提出解決方案代碼和工具函數文件。這包括指導和提示 Programmer 代理使用適當的工具函數來修復代碼，以解決模型中存在的問題。此模型品質涉及 Reviewer、Programmer 和模型檢查器的優化迴圈將反覆運算多次，直到 Checker 不報告任何錯誤，或者 Agent 無法自主解決問題。

為了確保代理能夠感知迴圈中全面的上下文資訊，我們實現了本地 loop-internal memory 模組來存儲 Programmer 和 Reviewer 之間的歷史交互。優化過程。此外，程式師和產品負責人共用的全域記憶體模組是用於存儲使用者的歷史聊天記錄和相應的代碼資訊。這允許代理在與人類對話期間獲得持續的上下文資訊，使整個框架能夠根據人工反饋改進和完善回應。通過這些多個優化迴圈，我們可以反覆運算指導 LLM 代理使用領域知識生成滿足特定設計品質、使用者意圖的 BIM 模型，和工程要求。

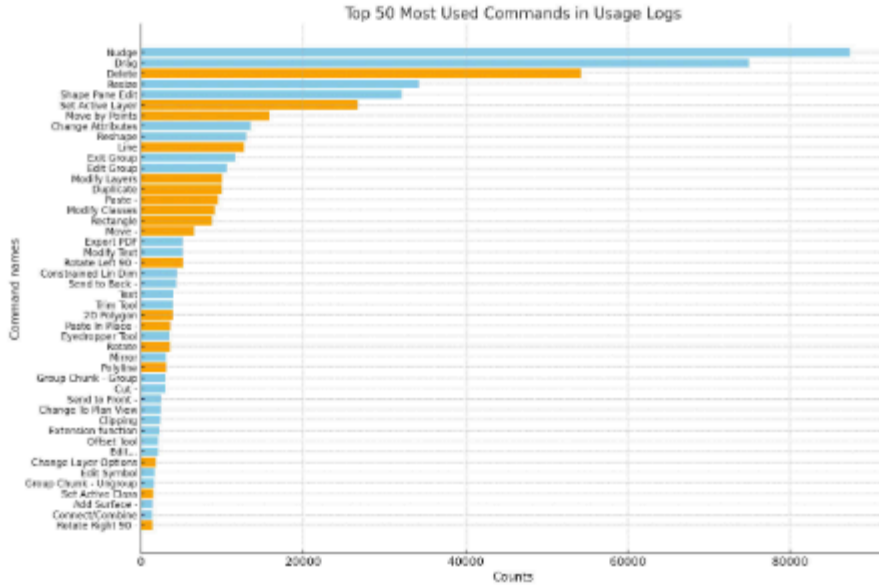


圖 2.BIM 創作軟體 Vectorworks 中最常用的 50 個命令（基於收集的大規模日誌數據集）

在以下小節中，我們將詳細描述擬議框架的幾個關鍵模組。

3.1 工具集

手動定義的工具函數基本上可以看作是公開給

LLM 的。由於 BIM 創作軟體的原生 API 通常是細粒度和低級的，因此每個工具本身都是封裝了組合不同可調用 API 函數以實現其功能的邏輯。這不僅避免了低級 API 調用的繁瑣和複雜性，但也包含特定的設計規則和工程邏輯，確保工具處理的建模任務的精度（Du 等人，2024b）。但是，設計通用工具有效應對各種建築場景的功能是一項重大挑戰。

為了解決這一困難，我們採用了定量和定性分析方法來確定工具函數來實現。我們通過檢查 BIM 創作軟體中的使用者紀錄檔開始調查了解人類設計人員在與軟體交互時最常使用哪些命令（工具）。我們利用了從設計軟體 Vectorworks 的 1,000 名匿名全球用戶那裡收集的一天日誌數據，包含 7 種語言的大約 2500 萬條記錄。在清理和過濾原始數據（Du et al. 2024a），提取出最常用的前 50 個命令，如圖 2 所示。

由於人類與軟體的交互方式與 LLM 代理的互動方式之間存在顯著差異 code，則由滑鼠作直接觸發的命令（如 drag、nudge 和 resize）佔據了很大比重的紀錄數據。此外，由於數據是完全匿名的，我們無法確定用戶的學科或使用場景。但是，仍然可以從這個龐大的數據集中觀察到建模過程中的一些常見步驟，如 Delete、Move、Duplicate、Set Working Layer 等。我們排除了以滑鼠為主的命令，並突出顯示了橙色部分是可以通過圖表中的 API 實現的通用建模命令，作為為代理構建工具功能。

另一方面，我們分析了內置的圖形程式設計工具 Marionette（類似於 Dynamo/Grasshopper）在 Vectorworks 中。事實上，這些可視化腳本平臺提供的節點（電池）通常是封裝的針對不同場景量身定製的底層 API 版本，作為更高級別、更直觀的程式設計設計人員的介面。軟體供應商根據預設節點的功能對預設節點進行分類，使其更容易供設計人員理解和使用它們。這些動機與我們的動機相似。鑒於我們創建常規 BIM 模型，我們主要是指「BIM」類別下的節點。

最後，我們還考慮了典型建築建模過程中所需的基本元件和步驟建築師，例如創建地板和牆壁、設置材料和立面圖等。通過全面綜合資訊從這三個角度出發，我們為智慧體設計了一組 26 個工具功能，如表 4 所示 - 附錄 I 中的 6 個，涵蓋 BIM 創作的重要方面，例如幾何建模和語義富集。由於 LLM 代理主要通過他們的描述來理解和使用這些工具，因此我們顯然使用結構化的 text 格式。

3.2及時改進和生成建築計劃

原始使用者提示 α 需要擴展和增強為更具體的表示形式，與建築設計。因此，產品負責人和架構師在這個階段的任務是至關重要的，因為他們確定最終生成模型的基本佈局和品質基線。提示範本 PArch 專為架構師如圖 3 所示。在這個範本中，我們首先定義 agent 的角色和任務，並嚴格指定輸出格式和內容，以滿足下游程式設計要求。然後，我們合併了 9 個基本架構規則和原則放入提示中（例如，元件配置、內部隔斷、建築物開放佈局等），指導 LLM 制定建築合理和結構綜合的建築計劃。

此外，我們採用小樣本學習方法（Brown 等人，2020 年）為 LLM 提供示例對話，以更好地引導它產生穩健的輸出。Architect 提示範本中的使用者輸入 α'

是產品負責人提供的原始使用者說明 α 的釋義。這是因為我們使用

函數調用機制來連接兩個代理：Architect 被包裹在函數層 F 中，使用者

You are an experienced architect who can design floor/building plans based on the user's needs. You will use your extensive architectural knowledge to expand and supplement the user's original description and ultimately express your design in structured text format.

Depending on the user's (and its needs, try to include in the output the starting and ending points of each wall, the location of windows and doors (offset relative to the start of the wall, boundaries of interior rooms/functional areas and the position and geometric details of other components required for a complete building.

Please refer to basic architectural rules, such like:

- Foundation: Ensure a solid foundation slab that can support the entire structure. The walls, slabs, and roof must be designed to distribute weight evenly to the foundation.
- Wall Configuration: Arrange walls to define the building's perimeter and internal spaces. Ensure that load-bearing walls are adequately spaced and placed to distribute the weight of the structure evenly. Set the wall elevations properly for each floor.
- Slab Design: Place slabs for each floor. They should be level and supported by the walls, providing stability and separating different floors.
- Roof Construction: Design the roof to cover the entire building, protecting it from weather elements.
- Window Placement: Install windows strategically to provide natural light and ventilation to rooms. Ensure window location are proportional to the room size.
- Door Placement: Position doors for easy access to different rooms and areas. Main entrance to the building should be prominent and easy to locate, with interior doors facilitating smooth movement.
- Interior Layout: Organize and define the interior room layout logically. Use interior walls to separate different functional rooms and ensure easy flow between them with a proportionally sized floor.
- Structural Integrity: Ensure all elements (walls, slabs, roof) are securely connected and stable.
- Compliance: Avoid clashing/overlapping building components, such as overlapping partitions between different areas and overlapping window and door locations. Adjacent rooms can share internal partitions. Rooms can also utilize exterior walls.

Make your design spatially and geometrically rational. Use millimeter units. Minimize other noise.

Here is a sample conversation:

```

User: I want to build an office building. I want the building to have 3 floors and the layout of each floor to be the same. Each floor has 6 rooms, 3 on each side, separated by a 3 meter wide corridor. Each room has a door and a window. The door to each room should be on the wall on one side of the corridor and the window should be on the outside wall of the building.
Architect: **3-Floor Office Building Design**

**Foundation**
- Rectangular foundation slab: 30000mm x 15000mm

**Ground Floor Plan**

1. **Perimeter Walls**
- Wall A: (0,0) to (30000,0)
- Wall B: (30000,0) to (30000,15000)
- Wall C: (30000,15000) to (0,15000)
- Wall D: (0,15000) to (0,0)

2. **Functional Areas**
Boundary in format [x1,y1,x2,y2] [x1,y1,y2,y2]
- Room 1: (0,0) to (10000,6000)
- Room 2: (10000,0) to (20000,6000)
- Room 3: (20000,0) to (30000,6000)
- Room 4: (0,6000) to (10000,12000)
- Room 5: (10000,6000) to (20000,12000)
- Room 6: (20000,6000) to (30000,12000)
- Corridor corridor: (0,6000) to (30000,6000)

3. **Location of Corridor Walls**
- Wall E: (0,6000) to (30000,6000)
- Wall F: (0,9000) to (30000,9000)

4. **Room Dividing Walls**
- Wall G: (10000,0) to (10000,12000)
- Wall H: (20000,0) to (20000,12000)
- Wall I: (10000,6000) to (10000,12000)
- Wall J: (20000,6000) to (20000,12000)

5. **Doors**
Insert an offset of each room door relative to the start of the corresponding wall:
- Room 1 door on corridor wall E: 5000
- Room 2 door on corridor wall E: 15000
- Room 3 door on corridor wall E: 25000
- Room 4 door on corridor wall F: 5000
- Room 5 door on corridor wall F: 15000
- Room 6 door on corridor wall F: 25000

6. **Windows**
Insert an offset of each room window relative to the start of the corresponding wall:
- Room 1 window on wall A: 5000
- Room 2 window on wall A: 15000
- Room 3 window on wall A: 25000
- Room 4 window on wall C: 5000
- Room 5 window on wall C: 15000
- Room 6 window on wall C: 25000

**Plot Floor Plan**
- Identical to Ground Floor Plan

**Second Floor Plan**
- Identical to Ground Floor Plan

**Roof Construction**
- Roof covering entire building: (0,0) to (30000,0) to (30000,15000) to (0,15000) to (0,0).

**Slab Design**
- Create slabs for each floor supported by perimeter and internal walls. Slabs covering entire floor area with same dimensions as foundation.

**Summary**
- Building dimensions: 30000mm x 15000mm x 3 floors.
- Each floor has 6 rooms, 3 on each side of a central corridor.
  
```

The user now provides the following instructions, please generate the plan as an architect. Let's think step by step.

User: extension

Architect:

圖 3. Architect 代理的提示範本。帶有“«»”的佔位符表示可以插入到範本中的動態內容。11Du, 2024 年 8 月 16 日

task 定義為輸入參數，Building plan 作為輸出。該描述記錄了 Architect 的責任，而這個函數是 Product Owner 唯一可以調用的工具。需要注意的是，該工具此處的函數調用與我們專為代碼生成任務設計的工具集無關。文本的生成建築規劃可以用方程 1 來概括：

$$\text{Building_Plan} \leftarrow F(\text{LLMArchitect}(\text{PAr ch}(\alpha))) \quad (1)$$

大多數主流 LLM 都經過微調以支援函數調用，使它們能夠返回結構化的 JSON-包含要執行的函數名稱和參數值的格式化回應（OpenAI 2024b; Mistral 2024a; 雙子座 2024）。函數調用的基本工作流程如下。當 Product Owner 收到使用者時說明，它會從 Prompt 上下文中推斷是否需要調用外部函數來完成任務。如果因此，它會生成一個 JSON 對象，明確指示函數名稱（Architect）和參數（釋義的使用者 needs）的 Needs）。我們在本地自動解釋 JSON 並調用 Architect 函數。收到任務后，Architect 生成相應的建築平面圖。然後，此資訊將作為附加上下文連接到 Product Owner 的提示符，指導它重新生成最終的標準文本回應。此設計的核心目標是為了讓產品負責人能夠根據 task，從而使兩個智慧體之間的交互更加智慧和高效。這樣，產品擁有者可以獨立處理簡單的任務，並利用架構師的專業知識處理複雜的任務，優化整個工作流程。

Product Owner 的提示範本 PPO 如圖 4 所示。產品負責人的角色是綜合各種上下文資訊，包括使用者的原始說明 α 、建築師的建築計劃、工具集中的函數用法描述 T，與全域記憶體中的使用者 Φ_{global} 的先前聊天歷史記錄模組，並在 prompt 範本中創建詳細而全面的需求文檔 α 增強。本文檔用作下游代碼生成任務的指南。我們利用提示中的 Thought（COT）技術（Wei et al. 2024），這需要 LLM 逐步推理，打破將 Building Modeling 任務簡化為利用不同工具功能的 sub-tasks 和 sub-logic。因此，整個使用者提示增強的過程可以用以下公式 2 表示：

$$\alpha \text{ 增強型} \leftarrow \text{LLMPProductOwner}(\text{PPO}(\alpha, T, \Phi_{\text{global}}) | \text{Building_Plan}) \quad (2)$$

3.3 BIM 模型生成的編碼

增強的使用者要求 α_{enhanced} 將輸入到程式師的提示範本 PCo 中，如圖所示如圖 5 所示。Programmer 代理需要僅使用

You are an experienced product owner who can instruct an AI programmer to write Python scripts in Vectorworks to accomplish user-specified tasks. You are a master at understanding API documentation, adept at breaking down the user's original instructions into subtasks and sub-logic based on the functions available, and able to express the user's description in a more rigorous and detailed manner. Please give detailed hints on coordinates and dimensions when instructing the programmer. Consult the Architect when you feel you need a building design plan or more architectural context to assist your instructions. If functional areas are given in the design plan, make sure to instruct the programmer to create them. You must refer to architectural knowledge to ensure that your content is spatially and geometrically rational. Use millimeter units. *Think step by step.* Minimize other prose.

For reference, the following are the API functions available to the programmer, try to give hints on how to use them effectively:

```
"""
<<Tool function names and descriptions>>
"""
```

Now, based on the conversation given below, please relay the user's instruction and complete your answer as a product owner. You don't need to write code, just guide the programmer step by step.

```
<<chat_history>>
User: <<task>>
Product Owner:
```

圖 4.Product Owner 代理的提示範本。帶有“<>”的佔位符表示可插入到範本中的動態內容。

You are an AI programmer that specialized in coding. Your job is to write Python code that invoke suitable pre-defined tool functions to implement product owner's requirement. Your code style is concise, efficient and accurate.

To help with that, you have access to a set of tools. Each tool is a Python function and has a description explaining the task it performs, the inputs it expects and the outputs it returns.

You should *think step by step* and write all your Python code in a single code block. You should avoid assuming in your code that variables are predefined. You can only give one reply for each conversation turn. Minimize other prose.

Accessible tools:

```
"""
<<Tool function names and descriptions>>
"""
```

In your code, you should only use the tool functions that are accessible to you, rather than attempting to invent new tools.

Use millimeter units. If necessary, you can import and use the Python standard library.

Now, based on the conversation given below, please complete your answer as a programmer:

```
<<chat_history>>
Product Owner: <<task>>
Programmer:
```

圖 5.Programmer 代理的提示範本。帶有“<>”的佔位符表示可以插入到範本中的動態內容。

工具集和內置的標準 Python 庫來完成產品擁有者指定的任務。這生成 Python 代碼的優勢在於 LLM 可以使用各種演算法邏輯。這比最近一項工作中基於 JSON 的函數調用更強大（Fernandes 等人，2024 年），受限於按順序執行單個功能，無法滿足任務複雜的邏輯需求例如建築建模。鑒於增強的要求包含豐富的上下文資訊，我們選擇利用代理的零樣本學習（Wei et al. 2022）能力，而無需提供範例。這種方法旨在允許 LLM 靈活地探索針對各種任務要求量身定製的不同代碼邏輯，而不是受到剛性示例的約束，從而最大限度地利用其預訓練知識。方程 3 說明瞭代碼生成過程，其中我們將工具資訊表示為 T ，並將與 Product 的歷史聊天記錄表示為 Owner 作為 Φ_{global} ：

$$\text{代碼} \leftarrow \text{LLMCoder}(\text{PCo}(\alpha_{\text{enhanced}}, T, \Phi_{\text{global}})) \quad (3)$$

自定義 Python 解釋器將在受控環境中執行生成的代碼。我們使用一個抽象語法樹（AST）來表示代碼，遍曆樹節點以評估每個 Python 表達式。這種方法允許我們自定義可用的語法和可調用的函數，同時啟用更精確的錯誤處理。這解釋器使用狀態字典來存儲和跟蹤代碼執行的結果，包括導入的包、定義的 function 物件以及變數名稱及其值。這為 Programmer 代理的 memory capability — LLM 可以使用和訪問之前對話中定義的變數和函數，也可以直接使用 Continue Previous 代碼。解釋器仍然可以通過檢索狀態字典來正確執行代碼，確保在整個會話中，在代碼級別維護全面的上下文。我們擴展了以前的工作（Du et al. 2024b）允許解釋器支援更多數據類型和高級語法，同時限制潛在的有問題的語法，如 while 語句。此外，解釋器只能評估工具集中的函數和 Python 內置庫（生成代碼中的自定義函數除外），阻止調用的任意第三方軟體包。

如果在代碼評估期間解釋器拋出異常 E ，則解決此錯誤將被視為新的 task 輸入。導致異常的代碼將用作新的歷史記錄資訊。雙錯誤和代碼將重新輸入到提示範本 PCo 中，以便程式師重新生成新代碼。如等式 3 所示，這個自反射修改迴圈將反覆運算最多 n 次，直到新生成的 $\text{coden}+1$ 沒有異常 E_{n+1} 。如果問題在 3 次嘗試內未得到解決，我們將中斷流程以尋求人工反饋。

$$\text{coden}+1 \leftarrow \text{LLMCoder}(\text{PCo}(E_n, T, \text{coden})) \quad \text{其中 } n \in [0, 3) \quad (4)$$

應該注意的是，在這個自反射迴圈中，不會對任何記憶體模組進行更新;只有

You are a senior code reviewer. You review Python code written by an AI programmer that calls specific APIs to generate buildings in Vectorworks. Your job is to propose solutions to the programmer for issues that arise in the generated building model, based on the API documentation. Issues are listed in the form of bullet points. Each issue contains a description and list of uuids of the corresponding elements in the generated building model. These uuids can be used to directly manipulate the related elements in Vectorworks via the API functions.

Here is the information you can refer to:

Code from the AI programmer:
«code»

Issues in the generated buildings:
«issues»

API documentation:
«Tool function names and descriptions»

Please provide your guidance below as a reviewer. Use millimeter units. *Think step by step.* Try to avoid duplicating the programmer's code, but rather suggest code fixes.

Reviewer:

圖 6.Reviewer 代理的提示範本。帶有“«»”的佔位符表示可以插入到範本中的動態內容。

state 字典將更新。在每次反覆運算中，Programmer 唯一可以看到的歷史資訊是代碼在上一步中失敗了。我們只存儲來自 Programmer 的回應，這些回應在記憶體中沒有錯誤模組。

3.4模型質量評估和反覆運算改進

成功執行的代碼將在 BIM 創作軟體中生成建築模型。雖然我們在早期階段使用了廣泛的 prompt 工程來指導 LLM 在空間和幾何上生產合理的結果，則該過程固有的隨機性仍可能導致生成的建築物出現缺陷。因此，使用確定性域特定規則來驗證和優化生成的 BIM 模型。我們雇傭了基於規則的模型檢查器來評估模型品質。根據（Solihin 和 Eastman 2015），我們定義了一個序列的規則涵蓋第 1 類到第 3 類，主要檢查元件之間的幾何衝突（例如，門和 windows overlap）、正確的語義屬性定義（例如，每個元件是否具有唯一的 GUID）以及空間佈局是否符合建築常識（例如，屋頂是否由牆壁支撐（floating））。所有規則的詳細文件可在附錄II中找到。模型中發現的問題將匯出為BCF檔。腳本用於自動從 BCF 中提取有用的資訊，即 name 和問題和相應規則的描述，以及關聯元件的 GUID。我們表示這個信息作為 I 並將其與生成選中模型的 code0 和工具集資訊 T 一起輸入到 Reviewer 代理的提示範本 PRev（參見圖 6），要求 Reviewer 提供β解決

問題。這個過程可以用方程 5 表示：

$$\beta \leftarrow \text{LLMReviwer}(\text{PRev}(I, T, \text{code})) \quad (5)$$

然後， β 作為輸入任務提供給 Programmer，以指導它生成代碼以解決衝突。此品質優化迴圈將反覆運算多次，直到檢查器不再發現任何錯誤。類似於自我反思迴圈中，如果問題在 3 次嘗試後仍然存在，我們將中斷迴圈並讓人工設計人員解決問題。在總結，我們的框架採用三重迴圈生成 BIM 模型並反覆運算提高其品質，這可以由演算法 1 總結。

演算法 1：用於模型生成和優化的三重迴圈

```
1: 當使用者給出指令時重複 $\alpha$                                 人工反饋迴圈
2:   調用 Architect 的條件： $Icondition \leftarrow \text{LLMProductOwner}(\text{PPO}(\alpha, T, \Phi_{\text{global}}))$ 
3:    $\text{Building\_Plan} \leftarrow Icondition \cdot F(\text{LLMArchitect}(\text{PArch}(\alpha)))$ 
4:    $\alpha$  增強型  $\leftarrow \text{LLMProductOwner}(\text{PPO}(\alpha, T, \Phi_{\text{global}}) | \text{Building\_Plan})$ 
5:   代碼生成： $\text{code0} \leftarrow \text{LLMCoder}(\text{PCo}(\alpha_{\text{enhanced}}, T, \Phi_{\text{global}}))$ 
6:   對於  $n = 0$  到 3，執行                                自反射迴圈
7:     代碼執行： $\text{Model} \leftarrow \text{Interpreter}(\text{coden})$ 
8:     if Exception En in coden then
9:       如果  $n = 3$  則
10:        停止並需要新的使用者輸入  $\alpha$ 
11:      還
12:        自更正： $\text{coden}+1 \leftarrow \text{LLMCoder}(\text{PCo}(\text{En}, T, \text{coden}))$ 
13:      end if
14:    還
15:      破
16:    end if
17:  end 為
18:  使用  $\alpha$ 、 $\alpha_{\text{enhanced}}$  和  $\text{coden}$  更新  $\Phi_{\text{global}}$ 
19:  使用  $\text{coden}$  更新  $\Phi_{\text{local}}$ 
20:  對於  $t = 0$  到 3 do                                品質優化迴圈
21:    模型檢查： $It \leftarrow \text{Checker}(\text{Modelt}, \text{Rules})$ 
22:    如果 Issues It 存在於 Modelt 中，那麼
23:      如果  $t = 3$  則
24:        停止、清除  $\Phi_{\text{local}}$  並需要人工干預
25:      還
26:        問題解決提案： $\beta t \leftarrow \text{LLMReviwer}(\text{PRev}(It, t, \Phi_{\text{local}}))$ 
27:        代碼修訂： $\text{codet}+1 \leftarrow \text{LLMCoder}(\text{PCo}(\beta t, T, \text{codet}))$ 
28:        重新進入自反射迴圈以執行  $\text{codet}+1$  生成  $\text{Modelt}+1$ 
29:        將無錯誤的  $\text{codet}+1$  儲存在  $\Phi_{\text{local}}$  中
30:      end if
31:    還
32:      斷開和清除  $\Phi_{\text{local}}$ 
33:    end if
end for35: 直到使用者閉合迴圈
```

4 原型實現

我們使用圖 7 所示的架構開發了一個互動式軟體原型，並將提議的框架導入到 BIM 創作軟體 Vectorworks 中。我們的實施基於 Vectorworks 的開源 Web 調色板外掛程式範本（Vectorworks 2024）並顯著擴展了以前的工作（Du et al. 2024b）以支援多代理工作流。Web 調色板的前端是使用 Vue.js 實現的，並在 Web 環境中運行基於 Chromium 嵌入式框架（CEF）構建，允許我們在 Vectorworks 中使用現代前端技術。Web 調色板的後端是一個 C++ 應用程式，支援定義和在 Web 框架中公開發布 JavaScript 函數，而實際邏輯是使用 C++ 實現的功能。

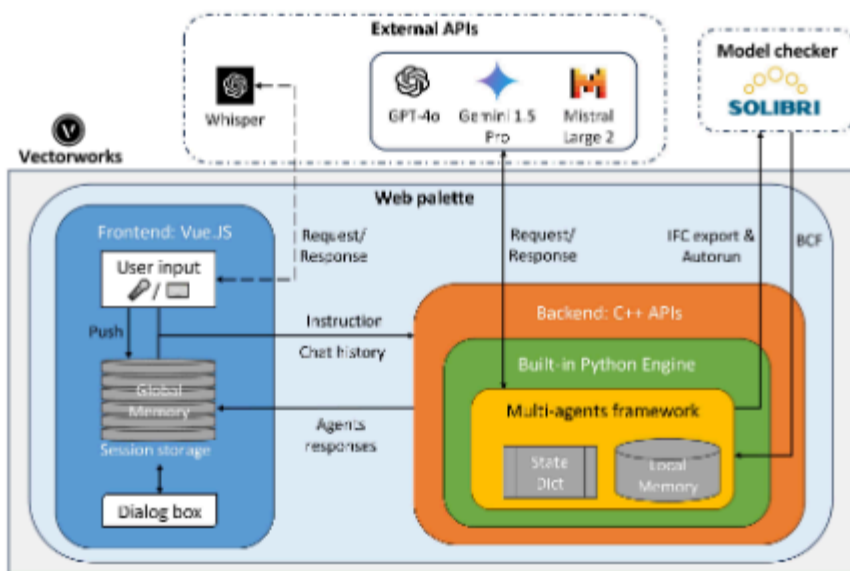


圖 7.基於 Web 調色板外掛程式範本的 Vectorworks 中已實現原型的軟體架構

由於我們的多代理框架完全基於 Python，因此我們調用 Vectorworks 的內置 Python 引擎 C++來執行我們的代碼，從而委託 JavaScript 實現。我們框架支持各種主流的 LLM，如 GPT-4o（OpenAI 2024a）、Gemini-1.5-pro（Google 2024）、以及最先進的開源模型 Mistral-Large-2（Mistral 2024b）。在該框架內，我們維護著 state 字典和本地記憶體模組來存儲 Python 調用之間的狀態和 agent 中。我們使用 Vectorworks 的 API 將生成的模型導出為 IFC 文件，然後使用 Solibri Autorun 命令自動啟動 Solibri Model Checker，執行檢查、

並輸出BCF。這種自動化使整個工作流程能夠完全自主運行。在 Solibri 模型中 Checker，根據附錄 II 中的規則文檔實施了 30 條規則，如圖 8 所示。

Name	Support Tag
• LUN-Ruleset	
• Class-Rules	
• 01_GeneralAspects	
§ Unique GUID values	SOL176/2.2
§ Layer Assignments	SOL230/1.1
§ Model Hierarchy	SOL176/2.2
§ Doors and Windows in same Room as Wall	SOL176/2.2
• 02_RequiredComponents	
§ Model Should Have Stairs	SOL114/2
§ Model Should Have Doors	SOL114/2
§ Model Should Have Windows	SOL114/2
§ Model Should Have Slabs	SOL114/2
§ Model Should Have Roofs	SOL114/2
§ Model Should Have Spaces	SOL114/2
• Class-Rules	
• Description Value should be set and filled with a GUID	
• Components should have a valid GUID	SOL230/1.1
• Wall Descriptions	SOL244/1.0
• Class-Rules	
• Object Intersections	
§ ItcSlab - ItcSlab Intersections	SOL115/3
§ ItcWall - ItcWall Intersections	SOL115/3
§ ItcWindow - ItcWindow Intersections	SOL115/3
§ ItcDoor - ItcDoor Intersections	SOL115/3
§ ItcRoof - ItcRoof Intersections	SOL115/3
§ ItcWall - ItcRoof Intersections	SOL115/3
§ ItcDoor - ItcWall Intersections	SOL115/3
§ ItcDoor - ItcWindow Intersections	SOL115/3
• Object Duplications	
§ ItcSlab - ItcSlab Duplications	SOL115/3
§ ItcWall - ItcWall Duplications	SOL115/3
§ ItcWindow - ItcWindow Duplications	SOL115/3
§ ItcDoor - ItcDoor Duplications	SOL115/3
§ ItcRoof - ItcRoof Duplications	SOL115/3
§ ItcWall - ItcRoof Duplications	SOL115/3
§ ItcDoor - ItcWindow Duplication	SOL115/3
• Object Adjacencies	
§ Stairs to Walls	SOL231/5.2
§ Roof to Walls	SOL231/5.2
§ Orphan Doors and Windows	SOL176/2.2
• Space Validation	
§ Space Area	SOL132/1.3

圖 8.在 Solibri Model Checker 中實現的規則

在前端，工作階段以新的使用者輸入開始。如果輸入是音訊，我們採用 Whisper 模型（Radford 等人，2022 年）將語音轉換為文本並填充輸入消息框。我們使用頁面會話存儲來存儲使用者輸入文字和代理的聊天記錄，在對話框中以不同的顏色。用戶可以通過重新載入頁面輕鬆啟動新工作階段。圖 9 顯示了開發的原型。使用者可以通過按兩下麥克風按鈕直接與代理聊天，直接在 Vectorworks 軟體。演示視頻可在補充材料中找到。

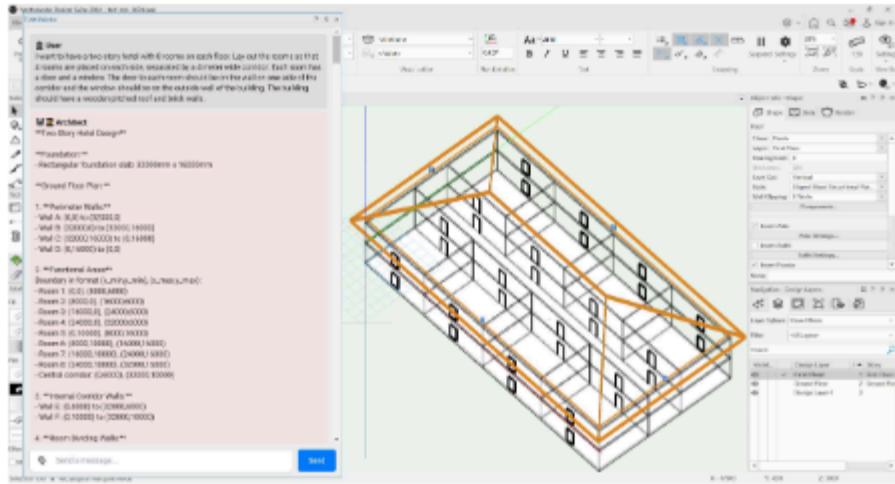


圖 9.無縫集成在 Vectorworks 中。用戶可以在內置聊天視窗中通過語音或文本給出說明。各個代理的回應在對話框中以不同的顏色顯示。生成的建築模型包含豐富的語義資訊，可以直接在 BIM 軟體中進一步編輯。演示視頻可以在補充材料中找到。

5 實驗和評估

本節專門對所提出的方法進行實驗評估。執行評估

通過使用測試使用者提示（說明）對建議的框架，並將生成的

各種 LLM 的結果，包括 GPT-4o（OpenAI 2024a）、Mistral-Large-2（Mistral 2024b）和 Gemini-1.5-Pro（谷歌 2024 年）。概念化了 10 個使用者提示，以全面測試生成能力和品質

從不同角度提出框架，如表 1 所示。表 2 總結了不同的

每個測試提示涵蓋的架構場景/要求，包括形狀、尺寸、空間等方面

特色、房間佈局、建築材料等，側重於必不可少的功能和美學元素

它們各自的目的（無論是住宅還是商業）。此外，通過故意離開一些建築物

要求中未指定，我們的目標是試驗框架在

開放式設置。鑒於生成模型固有的隨機性，每個測試提示都被輸入到每個 LLM 中

五次，總共得到 391 個 IFC 模型（包括優化過程的中間結果）。這

實驗基於此數據集，旨在報告具有統計學意義的結果。

表 3 顯示了模型檢查期間不同測試提示生成的最終模型的通過率。這些

費率是根據 Solibri 中實施的總共 30 個特定於域的檢查規則計算的，並反映了品質

生成的模型。“n”表示輸入此提示的次數。“平均通過率”記錄

表 1.測試使用者提示

不。	內容
1	我想建造一個兩層樓的酒店，每層有8個房間。房間每側四人一組，中間由4公尺寬的走廊隔開。每個房間都有一扇門和一扇窗。房間的門在牆壁的走廊一側，窗戶在建築物的外牆上。建築物應具有木製斜屋頂和磚牆。2創建尺寸為 10 x 6 米的四層住宅的基本 3D 模型。3創建總建築面積為 120 平方米的單層住宅。包括三間臥室、兩間浴室、一間廚房、
	和一個客廳。房屋應有一個木製斜屋頂，並至少包含四個窗戶和一個主入口門。使用混凝土牆。4創建具有三個連接部分的建築物。每個部分應為矩形（10m x 5m），牆壁高度為 3 m。這
	各段應由 5 米長的牆壁連接。添加樓板和覆蓋所有截面的連續混凝土斜屋頂。為每個部分添加門和窗戶。為牆壁選擇合適的材料5創建一個 3 層的 L 形房子，L 的每條腿長 8 米，寬 4 米。在 L 的拐角處放置一扇門，然後
	L 兩側各有一個視窗。我希望整個建築都是由木頭製成的。6設計一個具有複雜多邊形足跡（例如，六邊形）的建築。六邊形的每一邊應為 5 公尺。添加板
	地板和斜屋頂。包括一側的門和每側的窗戶。7建造一株具有矩形佔地面積（15m x 10m）、斜屋頂和兩層樓的住宅樓。通過擴展創建陽臺
	樓板從一樓的外牆向外延伸。為每層添加門窗。確保陽臺可從內部進入。8建造一座具有 20 米 x 20 米矩形底座的現代辦公樓。將牆高設置為 3 公尺。包括 4 個
	沿周邊的房間（每個5x5公尺）有一個中央開放空間。為每個房間添加門窗，併為建築物添加主入口門。9設計一株帶有 H 形底座的兩層公寓樓。每層由兩套公寓組成，每套公寓各有兩個房間。10創建一株水準截面為 10 米 x 30 米、垂直截面為 10 米 x 20 的 T 形單層建築
	米。通過在兩個部分的交匯處放置一扇門來連接這兩個部分。每個部分都有三個視窗。整個建築由混凝土製成。

表 2.測試提示中的使用者需求分類（“-”表示未指定）

不。	類型	地板	基礎形狀	屋頂	材料	房間	空間特性	建築空缺
1	旅館	2	矩形的	投	木 磚	16	4m 寬走廊	走廊側門，外牆窗
2	房子	4	矩形（10m x 6m）	-	-	-	-	-
3	房子	1	120 m2，形狀 未指定	投	木材、混凝土	7	-	至少是正門 四個視窗
4	建築	-	連接的矩形（10m x 5m）	投	-	-	連接截面 5m 長的牆壁	每個都有門和窗戶 部分
5	房子	3	L 形	-	木	-	L 的每條腿為 8 米 x 4 米	門在拐角處，窗戶開著 每側
6	建築	-	六角形	投	-	-	六邊形的每一邊都是 5 分鐘	單側門，窗戶 在彼此一側
7	住宅 建築	2	矩形（15m x 10m）	投	-	-	一樓陽臺	每個 地板，無障礙陽臺
8	辦公樓	-	矩形（20m x 20m）	-	-	4	中央開放空間、房間 沿周邊	每個都有門和窗戶 房間，正門
9	公寓 建築	2	H 形	-	-	8	2 間公寓設有2間客房 每	-
10	建築	1	T 形	-	混凝土	-	垂直截面：10m x20m，水平 截面： 10 米 x 30 米	交界處的門，三個 每個部分的視窗數

每個測試提示下相應 LLM 的平均通過率，以及所有 LLM 的總體平均通過率

測試案例。擬議框架下的三個 LLM 通常能夠生成高品質的 BIM 模型。

其中，GPT-4o 和 Mistral-Large-2 在所有測試中的平均通過率分別為 99.4% 和 99.2%

案例，而 Gemini-1.5-pro 僅達到 94.05% 的通過率。單獨檢查每個測試提示時，它會

很明顯，與其他兩個模型相比，Gemini 的性能明顯更加不一致，其中其指標在 66.67% 和 100% 之間波動，表明差異很大。可以考慮提示 3、4 和 5 “公認的挑戰”，因為沒有一個 LLM 能夠在所有五次運行中避免錯誤。此外，儘管 Mistral 是三款車型中尺寸最小的一款，它表現出了穩定性和非凡的推理能力，能力尤其令人印象深刻。

表 3.在測試提示下由不同主幹 LLM 生成的最終模型的規則通過率

主幹 LLM	n	提示 1	提示 2	提示 3	提示 4	提示 5	提示 6	提示 7	提示 8	提示 9	提示 10
GPT-4o 機器人	1	100.00%	100.00%	96.67%	100.00%	100.00%	100.00%	96.67%	100.00%	96.67%	100.00%
	2	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	96.67%
	3	100.00%	100.00%	100.00%	96.67%	100.00%	100.00%	96.67%	100.00%	100.00%	100.00%
	4	100.00%	100.00%	100.00%	100.00%	96.67%	100.00%	96.67%	100.00%	100.00%	100.00%
	5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	96.67%	100.00%
平均通過率		99.40%	100.00%	100.00%	99.33%	99.33%	100.00%	98.00%	100.00%	98.67%	99.33%
雙子座-1.5-Pro	1	73.33%	80.00%	96.00%	100.00%	80.00%	96.67%	100.00%	100.00%	96.67%	100.00%
	2	96.67%	100.00%	86.67%	96.67%	93.33%	100.00%	66.67%	100.00%	100.00%	96.67%
	3	100.00%	76.67%	100.00%	73.33%	90.00%	100.00%	96.67%	100.00%	96.67%	100.00%
	4	76.67%	96.67%	100.00%	100.00%	100.00%	100.00%	96.67%	100.00%	100.00%	100.00%
	5	93.33%	76.67%	86.67%	100.00%	96.67%	100.00%	100.00%	96.67%	100.00%	90.00%
平均通過率		94.05%	88.00%	86.00%	93.87%	94.00%	92.00%	99.33%	92.00%	99.33%	97.33%
Mistral-Large-2 (Mistral-大-2)	1	100.00%	93.33%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
	2	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	96.67%
	3	100.00%	100.00%	96.67%	100.00%	96.67%	100.00%	100.00%	100.00%	100.00%	100.00%
	4	100.00%	100.00%	100.00%	90.00%	96.67%	100.00%	100.00%	100.00%	100.00%	100.00%
	5	100.00%	100.00%	96.67%	96.67%	96.67%	100.00%	100.00%	100.00%	100.00%	100.00%
平均通過率		99.20%	100.00%	98.67%	98.67%	97.33%	98.00%	100.00%	100.00%	100.00%	99.33%

我們進一步評估了框架中提出的模型品質優化方法的有效性。與通過率是一個與類別相關的指標，我們選擇了Issue amount 作為更精細的實例級別度量。具體來說，雖然通過率提供了模型中存在的問題類型的廣泛概述（例如，如果模型未通過 IfcSlab-IfcSlab 交集規則，但通過了其他 29 條規則，則表示與問題相關的類型來防止板位置衝突），但 Issue 數量可以更深入地瞭解受影響的元件對的數量（例如模型中有多少對樓板實例存在定位衝突）。

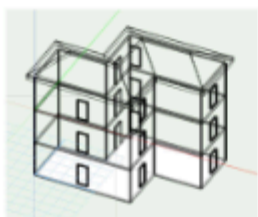
圖 10 顯示了三個折線圖，說明瞭 BIM 模型初始存在的平均問題數量生成階段，以及第一輪、第二輪和第三輪品質優化反覆運算之後。可以觀察到 GPT-4o 和 Mistral-Large-2 在充當 Reviewer 代理時，可以有效地反覆運算解決模型。平均問題數量總體上趨於減少，並最終收斂為該值較小，與表 3 中報告的結果一致。相比之下，Gemini-1.5-Pro 呈現上升趨勢，在一些測試提示符中，表示 Issue 數量增加。仔細查看 Gemini 的紀錄後

agent 中，我們發現 Gemini 經常傾向於完全重構之前的 Programmer 代碼，以生成一個新的 model 來嘗試解決問題，而不是按照 Prompt 範本中的說明創建代碼修復。這種方法可能會導致新舊模型的元件之間出現重複和衝突，從而導致發行量急劇增加。此外，過多的問題會對 LLM 作為審稿人，導致幻覺在反覆運算過程中累積。

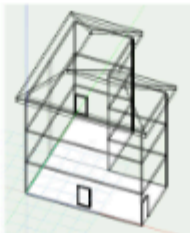


圖 10.不同 LLM 的品質優化迴圈的有效性

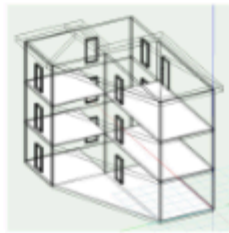
Create a 3-story L-shaped house with each ling of the L being 8 meters long and 4 meters wide. Place a door at the corner of the L and a window on each side of the L. I want the whole building to be made of wood.



GPT-4o



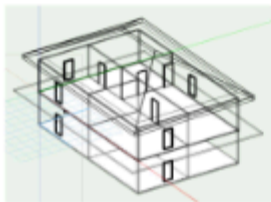
Mistral Large 2



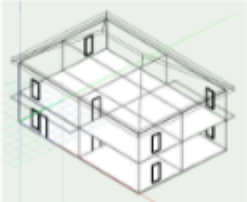
Gemini 1.5 Pro



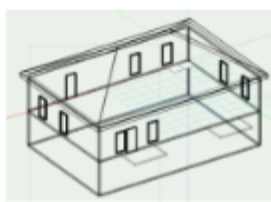
Construct a residential building with a rectangular footprint (15m x 10m), a pitched roof and two floors. Create balconies by extending the floor slab outwards from the exterior walls on the first floor. Add doors and windows to each floor. Make sure that the balconies are accessible from the inside.



GPT-4o



Mistral Large 2



Gemini 1.5 Pro

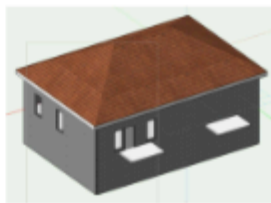
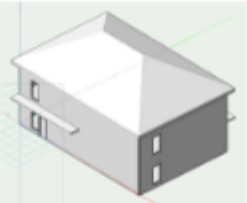
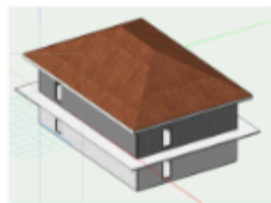


圖 11.Text2BIM 框架根據測試提示 5 和 7 生成的定性結果。其他提示生成的模型的可視化可以在附錄 III 中找到

圖 11 可視化了由擬議框架生成的部分建築模型。由於空間限制，生成的建築模型的完整清單可在附錄 III 中找到。通過仔細查看這些生成的模型，我們系統地評估了生成的建築物是否滿足

表 2.評估結果如圖 12 所示。無論使用哪種 LLM，由該框架通常能夠有效地滿足使用者的意圖。即使對於開放式指令不明確指定一些需求，我們的框架可以擴充和補全使用者的原始輸入。它利用 LLM 的預訓練架構知識和來自基於域規則的模型檢查器的反饋最終生產出在工程和建築方面都合理的建築。此外，它可以是觀察到建築物開口的合理佈置是一項需要高級空間理解的任務，構成對所有 LLM 來說都是一個挑戰。

圖 12. 根據用戶在測試提示中指定的要求評估生成的模型。0 表示說明中未指定要求，但 LLM 代理生成了相關結果。-0.5 表示代理未為未指定的要求生成任何相關結果。

實現的框架目前能夠在早期生成規則的、非彎曲的建築模型設計階段。為了將這種方法推廣到形狀不規則的建築物或更詳細的工程模型，需要為代理開發更複雜的工具，以顯著擴展現有的有限工具集。然而，在組織和管理大量的工具資訊及其相互依賴關係方面出現了挑戰。以便 LLM 可以有效地檢索有用的函數。知識圖譜和基於圖譜的 Retrieval Augmented 生成（RAG）技術可能會提供潛在的解決方案。

輔助功能等)和法規(例如,消防安全、區域要求等)。未來的工作可以探索如何將這些複雜的架構知識有效地集成到 LLM 中。

我們的實驗表明,LLM 代理可以自動將模型內的衝突解決到有限的範圍。雖然本模組不是本研究的主要重點,我們在這個方向上的初步探索為相關領域的研究提供了一種新的技術方法。這考慮到當前對自動衝突解決的研究主要集中在使用優化演算法(Wu 等人,2023年)、經典機器學習(Harode 等人,2024 年)或強化學習(Harode 等人,2022 年)。儘管取得了這些進步,但基於 LLM 代理的衝突解決方法仍然具有重大限制。圖 13 總結了品質優化過程中遇到的一些代表性場景

圈。第一個常見故障 (a) 涉及代理嘗試重寫代碼以創建新模型,從而導致由於新模型元件和現有模型元件之間的衝突而增加的發行量。在情景 (b) 中,上面的兩個初始模型的樓板具有重疊和嵌套的牆、門和窗。在這種高度複雜的情況下,LLM 代理,僅依賴代碼和檢查器反饋(規則/問題描述)來獲取上下文資訊,無法解決所有問題,並且容易產生幻覺。代理在這裡採用的策略涉及刪除部分相關樓層的牆壁。雖然此作可以減少模型中的總體問題數量,但它會帶來損害建築物的結構完整性。但是,當前的代理只能感知 1D 文字中的資訊,並且還不能以這種方式理解 3D 空間。場景 (c) 說明瞭一個成功的案例,其中代理正確調整浮動屋頂的高度,使其與頂層的牆標高對齊。總體而言,LLM 表現良好對於確定性解決方案(通常是第 1 類規則,例如“在model -> create 中沒有定義空間”的直觀問題 space”)。但是,它們經常在需要更高層次空間理解且具有開放式的複雜問題上失敗

解決方案(通常是第 3 類規則,例如「兩堵隔牆相交 ->要移動哪堵牆,以及在哪一堵牆中移動方向?儘管我們的框架允許用戶指導 LLM 執行適當的問題解決作

通過對話或手動繼續編輯軟體中生成的 BIM 模型,未來的研究將優先考慮增強 LLM 的空間理解能力,以朝著自主衝突解決系統邁進。

鑒於我們的方法基於提示工程技術生成 3D 模型的代碼表示,它不需要對 LLM 進行微調。這與傳統的文本到 3D 方法有著根本的不同,後者通常需要構建 3D 數據集進行訓練。常用指標,例如倒角距離 (CD) 和交併比 (IoU) 主要側重於評估點雲/體素模型的幾何精度。如這些指標不適用於我們的數據表示方法,我們建議使用 Domain-Specific 的 pass rate 規則檢查作為定量指標來評估生成的 BIM 模型。雖然此方法可以驗證生成的模型在結構上是完整的,並且在架構方面是合理的,但它的局限性在於 Model Checkers 提供的規則無法評估生成的建築物是否與 Abstract 和 Dynamic 一致以自然語言指令表達的使用者意圖(例如,“H 形房子”、“沿建築物佈置房間”

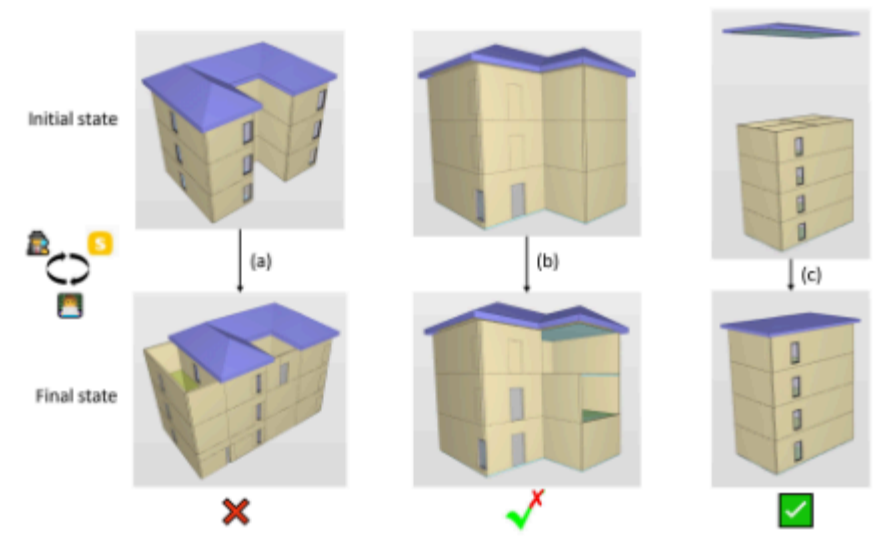


圖 13.在品質優化迴圈中遇到的幾個典型案例

perimeter“等）。目前，我們仍然依靠人工審核來確定模型是否符合預期指示。未來的研究可以利用這項工作產生的數據來開發新的基準數據集，以及指標，從而實現對使用者意圖的自動化、數據驅動型評估。

7 結論

我們介紹了 Text2BIM，這是一個基於 LLM 的多代理協作框架，可在來自自然語言描述的 BIM 創作軟體。本研究的主要發現和貢獻如下遵循：

- 與以前專注於生成建築物的 3D 幾何表示的研究不同，我們的框架能夠生成具有內部布局、外部圍護結構和語義資訊的本地 BIM 模型。
- 我們建議使用與 BIM 創作軟體 API 交互的命令式代碼來表示 3D 建築模型。通過採用快速工程技術，多個 LLM 代理協作開發代碼，而無需微調，從而節省計算資源。
- 創新地將特定領域的基於規則的模型檢查器集成到框架中，以指導 LLMs 生成架構和結構合理的結果。所提出的品質優化迴圈表明，LLM 代理可以根據檢查器的文本反饋反覆運算解決 BIM 模型中的衝突。

- 已經進行了廣泛的實驗，以全面評估所提出的框架內不同模組的性能，包括對框架內三個開源/閉源 LLM 的比較分析，這驗證了我們方法的普遍性和有效性。

- 開發了一個互動式軟體原型，將建議的框架集成到 BIM 創作工具 Vectorworks 中，展示了在設計過程中通過聊天建模的創新可能性。

我們相信，所提出的方法可以擴展到更廣泛的用例，而不僅僅是模型代，特別是如果為 LLM 代理開發了更專業的工具以供使用。我們希望讀者能找到從中汲取靈感，並探索使用 LLM 來解決我們領域內的更多挑戰。

8 數據可用性聲明

支援本研究結果的一些數據和模型可從通訊作者處獲得合理的請求。

9 確認

這項工作由 Nemetschek Group 資助，對此表示感謝。我們衷心感謝這些數據以及 Vectorworks，Inc. 提供的許可支援。

10 補充材料

1. 演示視頻

附錄 I. 工具集文件

表 4.工具集 (a)

工具功能名稱	描述
create_story_layer	此工具用於創建新的故事圖層。將在給定的高程處創建新圖層。創建新的樓層圖層后，它將成為活動圖層。所有新的建築元素都將在當前啟動樓層上創建。輸入： - layer_name：str，新故事的唯一名稱。 - elevation：float，新樓層相對於地面的高度。 - floor_index：int，新 floor 的索引。應從 1.Return 開始： - str，新故事層的layer uuid。
set_active_story_layer	此工具用於將具有給定名稱的故事圖層設置為 active。活動故事圖層是在其中創建新元素的圖層。輸入： - layer_name：str，要設置為活動層次的名稱。返回： - str，活動層的layer uuid。
create_functional_area	此工具用於在指定圖層上創建概念功能區域。該區域是根據定義房間邊界的頂點清單創建的。通常，首先創建功能區域來定義建築物的內部佈局，然後通過在邊界處放置牆壁來分隔房間。輸入： - vertices：元組清單，每個元組代表定義房間邊界的頂點的 2D 座標。 - 名稱：str，房間/功能區域的名稱。 - layer_uuid：str，將在其中創建空間的故事圖層的 UUID。返回： - str，創建的房間/功能區的 UUID。
create_wall	此工具用於在指定圖層上創建牆。默認情況下，牆的創建bottom_elevation為 0，atop_elevation 相對於此層次為 3000。輸入： - st_pt：tuple，牆起點的 2D 座標。 - ed_pt：tuple，牆端點的 2D 座標。 - layer_uuid：str，將在其中創建牆面的樓層圖層的 UUID。返回： - str，新創建的 Wall 的 UUID。
set_wall_thickness	此工具用於設置壁的厚度。輸入： - uuid：str，wall 物件的 uuid。 - thickness：float，牆的新厚度。返回： - str，已修改的 wall 物件的 uuid。
set_wall_elevation	此工具用於設置牆壁的頂部/底部標高。減去這兩者是牆本身的高度。輸入： - uuid：str，wall 物件的 uuid。 - top_elevation：float，從牆壁頂部到最初創建牆壁的樓層圖層的垂直距離。 - bottom_elevation：float，從牆壁底部到最初創建牆壁的樓層圖層的垂直距離。返回： - str，已修改的 wall 物件的 uuid。
get_wall_elevation	此工具用於獲取牆的頂部和底部標高。減去這兩者是牆本身的高度。輸入： - uuid：str，wall 物件的 uuid。返回： - top_elevation：float，從牆壁頂部到最初創建牆壁的樓層圖層的垂直距離。 - bottom_elevation：float，從牆壁底部到最初創建牆壁的樓層圖層的垂直距離。
get_wall_thickness	此工具用於獲取牆的厚度。輸入： - uuid：str，wall 物件的 uuid。返回： - thickness：float，壁的厚度。

表 5.工具集 (b)

工具功能名稱	描述
set_wall_style	此工具用於設置牆壁的樣式。輸入： - uuid：str，wall 物件的 uuid。 - style_name：str，樣式的名稱。以下牆樣式名稱可用：["Exterior Concrete Wall"，"Exterior WoodWall"，"Exterior Brick Wall"，"Interior Concrete Wall"，"Interior Wood Wall"，"Interior Brick Wall"]返回： - str，已修改的 wall 物件的 uuid。
add_window_to_wall	此工具用於將視窗添加到牆壁。一旦窗戶被添加到牆壁上，它就是牆壁的一部分，並將與牆壁一起移動/複製/旋轉。輸入： - wall_uuid：str，視窗將要添加到的 Wall 物件的 UUID。 - window_elevation：float，窗戶從牆底部開始的標高。 - window_offset：float，視窗與牆壁起點的偏移量。 - window_name：str，需要添加的視窗物件名稱。返回： - str，已添加到 Wall 的 Window 物件的 UUID。
add_door_to_wall	此工具用於將門添加到牆壁。一旦門被添加到牆壁上，它就是牆壁的一部分，並將與牆壁一起移動/複製/旋轉。輸入： - wall_uuid：str，門將添加到的牆壁物件的 UUID。 - door_elevation：float，門從牆底部開始的高度。 - door_offset：float，門與牆壁起點的偏移量。 - door_name：str，需要添加的門物件的名稱。返回： - str，已添加到牆體的門物件的 UUID。
move_obj	此工具用於移動元素。它只能在放置它的圖層內移動給定的元素，而不能複製它。輸入： - uuid：str，要移動的元素的唯一 uuid。 - xDistance：浮點型，在 x 方向上的移動距離。 - yDistance：浮點型，在 y 方向上的移動距離。 - zDistance：浮點型，在 z 方向上的移動距離。返回： -沒有
delete_element	此工具用於刪除元素或元素清單。無法刪除故事圖層。輸入： - uuid：str 或字串清單，需要刪除的元素的唯一 uuid。返回： -沒有
find_selected_element	此工具用於在當前活動樓層圖層中查找所選元素。如果未找到選定的元素，它將返回一個空清單。輸入： - NoneReturn： - list of str，所選元素的 UUID。
create_polygon	此工具用於使用其振點在指定故事圖層上創建多邊形。輸入： - vertices：元組清單，每個元組代表多邊形一個頂點的 2D 座標。 - layer_uuid：str，將在其中創建多邊形的故事圖層的 UUID。返回： - str，創建的多邊形的 UUID。
get_polygon_vertex	此工具用於在多邊形的頂點陣列中給定索引處獲取所需的頂點。輸入： - uuid：str，多邊形物件的 uuid。 - at：int，所需頂點的索引。返回： - tuple，多邊形所需頂點的 2D 座標。
get_vertex_count	此工具用於獲取多邊形中的振點數。輸入： - uuid：str，多邊形物件的 uuid。返回： - int，輸入多邊形中的頂點數。

表 6.工具集 (c)

工具功能名稱	描述
create_slab	此工具用於從指定圖層上的多邊形輪廓創建板。輸入： - profile_id：str，多邊形物件的 UUID，用於確定板的輪廓。 - layer_uuid：str，將在其中創建 slab 的故事圖層的 UUID。返回：
set_slab_height	- str，創建的 slab 的 UUID。 此工具用於設置板的高度（標高）。輸入： - slab_id：str，slab 物件的 uuid。 - height：float，樓板相對於最初創建樓板的樓層圖層的高度。返回：
get_slab_height	- str，修改後的 slab 的 UUID。 此工具用於獲取板的高度（標高）。輸入： - slab_id：str，slab 物件的 uuid。返回：
set_slab_style	- float，樓板相對於最初創建樓板的樓層圖層的高度。 此工具用於設置板的樣式。輸入： - slab_id：str，slab 物件的 uuid。 - style_name：str，樣式的名稱。返回：
duplicate_obj	- str，修改後的 slab 的 UUID。 此工具用於將元素複製到指定層次。請注意，在複製包含門和窗的牆時，其中的門和窗也會被複製。故事圖層不能複製。輸入： - element_uuid：str，要複製的元素的唯一 uuid。 - layer_uuid：str，將放置副本的故事圖層的 UUID。 - n：int，要製作的副本數。返回：
rotate_obj	- list of str，副本的 UUID 清單。建議使用此清單進一步作副本。 此工具用於旋轉元素。輸入： - uuid：str，要旋轉的元素的唯一 uuid。 - angle：float，旋轉元素的角度（以度為單位）。 - center：tuple，旋轉中心的 2D 座標。默認情況下，它是元素的中心。（選擇）返回：
create_pitched_roof	- str，旋轉元素的 UUID。 此工具用於根據指定圖層上的多邊形剖面創建斜屋頂。輸入： - profile_id：str，多邊形物件的 uuid，用於確定屋頂的輪廓（底部）。 - layer_uuid：str，將在其中創建屋頂的故事圖層的 UUID。 - slope：浮點型，屋頂的坡度，以度為單位。它不能小於 5。 - eave_overhang：float，屋頂的屋簷懸垂。 - eave_height：浮點型，屋頂相對於指定圖層的標高。通常是此樓層的牆壁高度。 - roof_thickness：float，屋頂的厚度。返回：
set_pitched_roof_attributes	- str，創建的屋頂的 UUID。 此工具用於設置斜屋頂的新屬性。可以選擇性地輸入需要更改的屬性。輸入： - roof_id：str，屋頂物件的 uuid。 - slope：float，屋頂的坡度，以度為單位（可選）。 - eave_overhang：浮子，屋面屋簷懸垂（可選）。 - eave_height：float，最初創建屋頂的樓層圖層的屋頂高度（標高）（可選）。 - roof_thickness：float，屋頂的厚度（可選）。返回：
set_pitched_roof_style	- str，修改後的屋頂的 UUID。 此工具用於設置斜屋頂的樣式。輸入： - roof_id：str，屋頂物件的 uuid。 - style_name：str，樣式的名稱。可用：["Low Slope Concrete w/ Rigid Insulation", "Sloped Wood Struct InsulFlat Clay Tile"]返回：
	- str，修改後的屋頂的 UUID。

表 7：

第 1 類規則	
範圍	該規則檢查是否所有元件都具有唯一的 GUID。
Solibri 規則 ID	SOL/176/2.2
元件	任何元件
所需解析度	優化那些尚未通過此規則的元件的 GUID。
範圍	該規則檢查模型是否具有由 IfcSite、IfcBuilding 和 IfcBuildingStorey 實例。
Solibri 規則 ID	SOL/176/2.2
元件	空間劃分元素
所需解析度	創建適當的空間容器並相應地分配元件。
範圍	該規則檢查所有門和窗是否與包含牆位於同一樓層。
Solibri 規則 ID	SOL/176/2.2
元件	門窗
所需解析度	為每個受影響的門或窗重新分配空間關聯。
範圍	該規則將檢查每個元件是否附加了層資訊。
Solibri 規則 ID	SOL/230/1.1
元件	除開孔外的任何元件
所需解析度	將圖層資訊添加到受影響的零部件。
範圍	該規則檢查模型中是否存在某些元件（例如，牆壁、門、窗、板、屋頂、空間）。
Solibri 規則 ID	SOL/11/4.2
元件	任何元件
所需解析度	根據使用者的輸入創建缺失的元件。

表 8：

第 2 類規則	
範圍	該規則檢查是否設置了所有建築元件的 description 值，並且該值符合替換為 UUID 的模式。
Solibri 規則 ID	SOL/244/1.0
元件	任何物理元件
所需解析度	將 Vectorworks 內部 ID 設置到那些元件的描述欄位中，這些元件沒有通過了這條規則。

表 9：

第 3 類規則	
範圍	該規則檢查一個元件是否與其他元件相交。
Solibri 規則 ID	SOL/1/5.0
元件	任何物理元件
所需解析度	重新定位元件以避免相交
範圍	該規則檢查兩個元件是否彼此重複。
Solibri 規則 ID	SOL/1/5.0
元件	任何物理元件
所需解析度	刪除其中一個元件
範圍	該規則檢查兩個元件之間的連接： 屋頂必須與最上層的牆壁相連。 樓板必須連接到支撐牆。
Solibri 規則 ID	SOL/23/5.2
元件	屋頂、樓板和牆壁
所需解析度	將屋頂/板移動到支撐牆的頂部
範圍	該規則檢查模型不包含任何孤立的門或窗（門或 window 的 URL 中，它與任何牆都沒有關係）。

下一頁繼續

表 9：(續)

第 3 類規則	
Solibri 規則 ID	SOL/176/2.2
元件	門窗
所需解析度	拆除孤立的門或窗

附錄 III.生成模型的視覺化

I want to build a two-story hotel with eight rooms on each floor. The rooms are arranged in groups of four on each side, separated by a 4-meter-wide corridor in the middle. Each room has a door and a window. The doors of the rooms are on the corridor side of the wall, and the windows are on the outside wall of the building. The building should have a wooden pitched roof and brick walls.

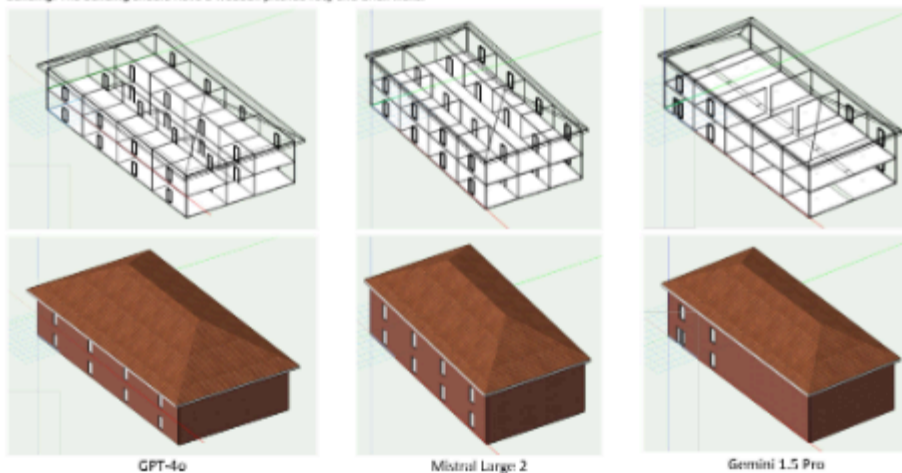


圖 14.不同 LLM 通過 proposedframework 根據相應的文本描述（Prompt Nr.1）生成的線框/渲染模式的建築模型。

Create a building with three connected sections. Each section should be a rectangle (10m x 5m) with walls of 3 meters in height. The sections should be connected by 5-meter-long walls. Add slabs and a continuous, concrete-pitched roof that covers all sections. Add doors and windows to each section. Choose the right material for the wall.

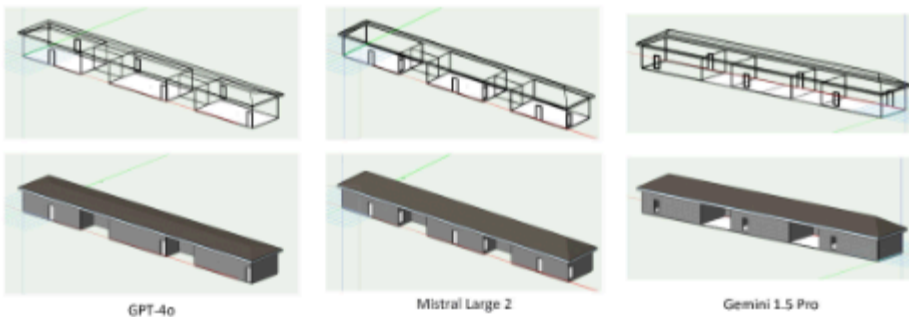


圖 15.由不同的 LLM 通過 proposedframework 根據相應的文本描述（Prompt Nr.4）生成的線框/渲染模式下的建築模型。

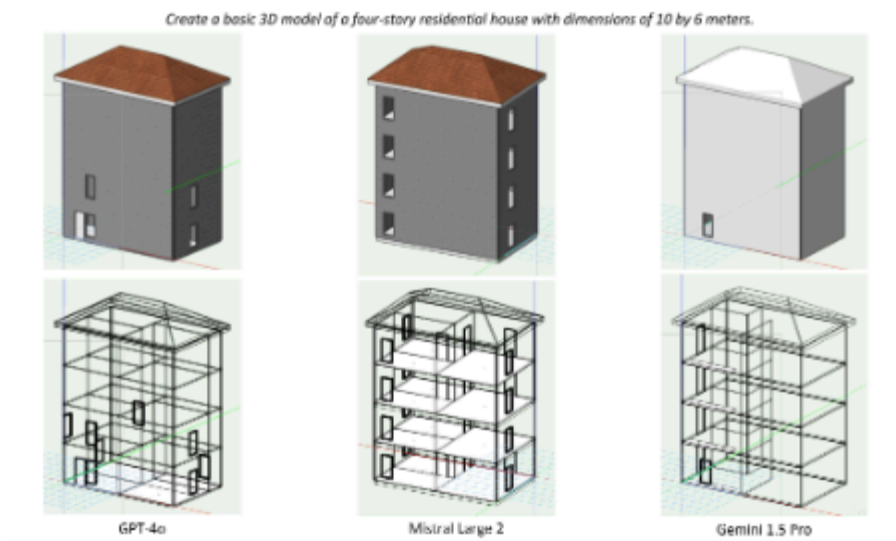


圖 16.不同 LLM 通過 proposedframework 根據相應的文本描述（Prompt Nr.2）生成的線框/渲染模式下的建築模型。

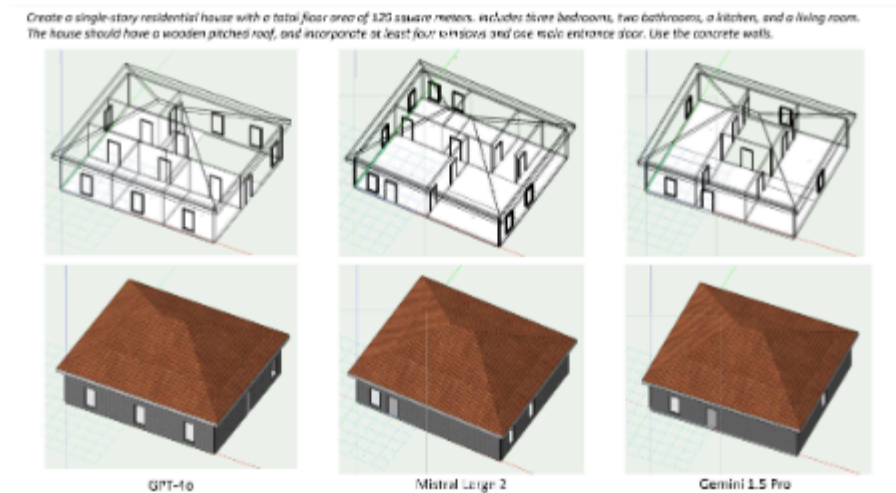
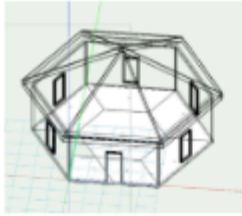
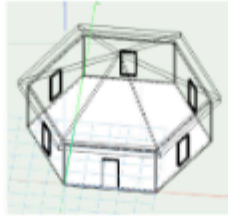


圖 17.不同 LLM 通過 proposedframework 根據相應的文本描述（Prompt Nr.3）生成的線框/渲染模式下的建築模型。

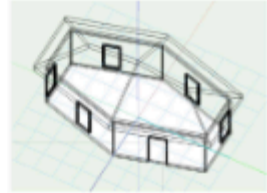
Design a building with a complex polygonal footprint (e.g., hexagon). Each side of the hexagon should be 5 meters. Add a slab for the floor and a pitched roof. Include a door on one side and a window on each of the other sides.



GPT-4o



Mistral Large 2



Gemini 1.5 Pro

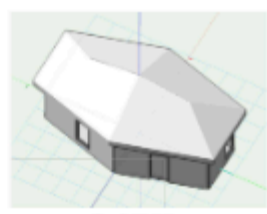
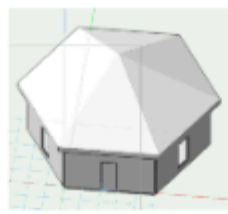
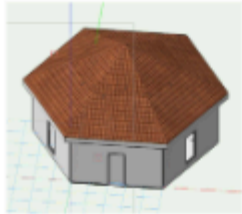
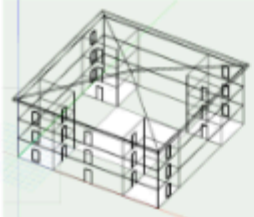
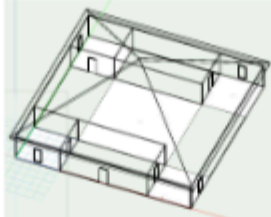


圖 18.由不同的 LLM 根據相應的文本描述（Prompt Nr.6）通過 proposedframework 生成的線框/渲染模式下的建築模型。

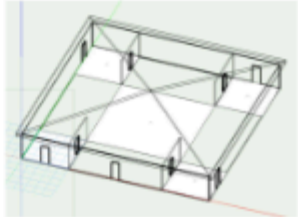
Construct a modern office building with a rectangular base of 20 meters by 20 meters. Set the wall height to 3 meters. Include four rooms (5x5 meters each) along the perimeters, with a central open space. Add doors and windows to each room and a main entrance door to the building.



GPT-4o



Mistral Large 2



Gemini 1.5 Pro

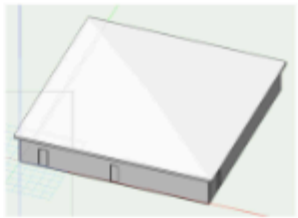
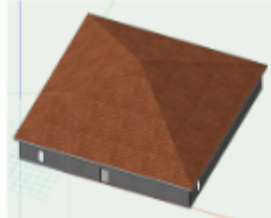
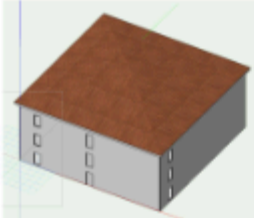


圖 19.由不同的 LLM 根據相應的文本描述（Prompt Nr.8）通過 proposedframework 生成的線框/渲染模式下的建築模型。

Design a two-story apartment building with an H-shaped base. Each floor consists of two apartments with two rooms each.

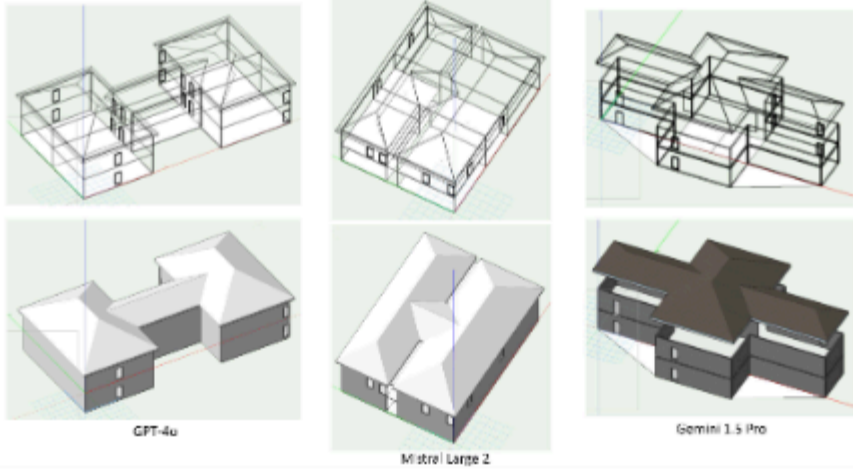


圖 20.不同 LLM 通過 proposedframework 根據相應的文本描述 (Prompt Nr.9) 生成的線框/渲染模式的建築模型。

Create a T-shaped, single-story concrete building with a horizontal section of 10 meters x 30 meters and a vertical section of 10 meters x 20 meters. Connect the two sections by placing a door at their junction. Each section has three windows. The entire building is made of concrete.

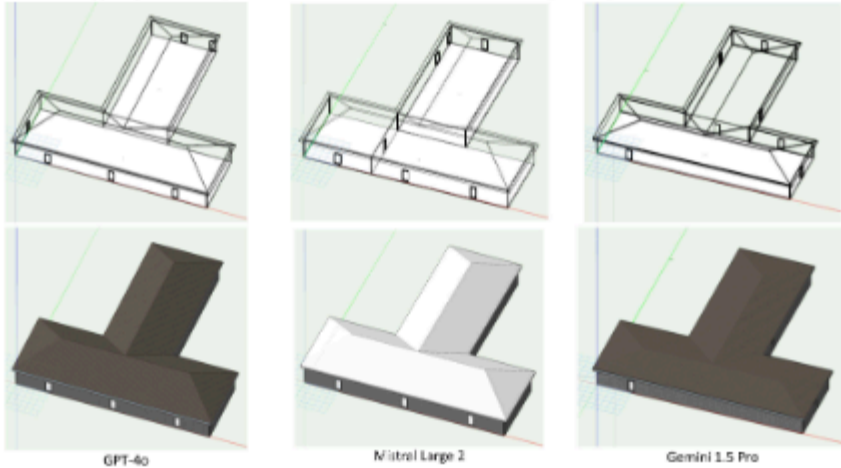


圖 21.不同 LLM 通過 proposedframework 根據相應的文本描述 (Prompt Nr.10) 生成的線框/渲染模式的建築模型。

引用

Borrmann, A., König, M., Koch, C. 和 Beetz, J. (2018)。“建築資訊建模：為什麼？什麼？怎麼做到的？

建築資訊模型 - 技術基礎和行業實踐, A. Borrmann、M. König、C. Koch、
和 J. Beetz 編輯, 第 1 卷, 施普林格, 1-24。

布朗, TB, 曼恩, B., 萊德, N., 蘇比亞, M., 卡普蘭, J., 達里瓦爾, P., 尼拉坎坦, A., 希亞姆, P., 薩斯特裡, G., 阿斯凱爾,

A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, DM, Wu, J., Winter,

C., 黑塞, C., 陳, M., 西格勒, E., 利特文, M., 格雷, S., 國際象棋, B., 克拉克, J., 伯納, C., 麥克坎德利什, S., 拉德福德, A., 西格勒, E., 利特文, M., 格雷, S., 國際象棋, B., 克拉克, J., 伯納, C., 麥克坎德利什, S., 拉德福德, A., 西格勒, E., 利特文, Sutskever, I. 和 Amodei, D. (2020)。“<<https://arxiv.org/abs/2005.14165>>, 語言模型是少數機會的學習者。

陳, J., 邵, Z. 和胡, B. (2023)。“從文本生成室內設計：一種基於擴散模型的新方法
高效的創意設計。建築物, 13 (7)。

de Miguel Rodríguez, J., Villafañe, ME, Piškorec, L. 和 Sancho Caparrini, F. (2020)。“幾何的生成
具有深度變分自動編碼器的建築物類型的插值。設計科學, 6, e34.

董Q., 李, L., 戴, D., 鄭, C., 馬, J., 李, R., 夏H., 徐, J., 吳, Z., 張, B., 孫, X., 李, L., 和隋, Z.
(2024)。“關於情境學習的調查, <<https://arxiv.org/abs/2301.00234>>。

杜, C., 鄧, Z., 努西亞斯, S. 和博爾曼, A. (2024a)。“Towards commands recommender system in bim authoring (面向 BIM 創作中的命令推薦系統)
使用變壓器的工具。第 31 屆國際工程智慧計算會議 (EG-ICE) 論文集 (7 月)。

Du, C., Nousias, S. 和 Borrmann, A. (2024b)。“使用大型語言模型實現 BIM 創作工具的 copilot
基於智慧人機交互的代理。第 31 屆智慧計算國際會議論文集
工程 (EG-ICE) (7 月)。

伊士曼, C., 敏李, J., suk Jeong, Y. 和 kook Lee, J. (2009)。“基於規則的建築設計自動檢查。”
建築自動化, 18, 1011–1033.

Ennemoser, B. 和 Mayrhofer-Hufnagl, I. (2023)。“通過開發一種新穎的方法跨多尺度數據集進行設計
到 3dgans。國際建築計算雜誌, 21 (2), 358-373。

費爾南德斯, D., 加格, S., 尼克爾, M. 和古文, G. (2024 年)。“一個由 gpt 提供支援的助手, 用於與
構建資訊模型。建築物, 14 (8)。

福克斯, S., 維特布羅克, M., 迪米亞迪, J. 和阿莫爾, R. (2022 年)。“建築法規的神經語義解析
合規性檢查。IOP 會議系列：地球與環境科學, 1101, 092022。

雙子座 (2024)。“使用 gemini api 進行函數調用簡介, <<https://ai.google.dev/gemini-api/docs/function-calling>>。
訪問時間：2024年7月16日。

谷歌, GT (2024年)。“Gemini 1.5：在數百萬個上下文標記中解鎖多模態理解，
<<https://arxiv.org/abs/2403.05530>>。

石墨軟體 (2024)。“Archicad AI 可視化器, <<https://graphisoft.com/solutions/innovation/archicad-ai-visualizer>>。交流-

於 2024 年 6 月 27 日停止。

郭 T.、陳 X.、王 Y.、張 R.、裴 S.、Chawla, NV、Wiest, O. 和 Zhang, X. (2024 年)。“大語種基於模型的多智慧體：進展和挑戰調查，<<https://arxiv.org/abs/2402.01680>>。

Harode, A.、Thabet, W. 和 Gao, X. (2022 年)。“一種集成的監督強化機器學習方法自動衝突解決。679–688.

Harode, A.、Thabet, W. 和 Gao, X. (2024 年)。“開發機器學習模型來預測衝突解決選項。土木工程計算雜誌, 38 (2), 04024005.

He, Z., Wang, Y.-H. 和 Zhang, J. (2023)。“集成 BIM 和擴散模型的生成式結構設計，<<https://synthical.com/article/bb21e837-1ed0-4489-8a33-768e6d0882fb>> (10)。

洪, S., 朱戈, M., 陳, J., 鄭X., 程Y., 張, C., 王J., 王, Z., 邱S. K. S., 林, Z., 周, L., 林, Z., 周, L., 陳, J., 鄭X., 程Y., 張, C., 王J., 王, Z., 邱S. K. S., 林, Z., 冉, C., 肖, L., 吳, C. 和施米德胡伯, J. (2023 年)。“Metagpt：用於多代理協作的Meta程式設計框架，<<https://arxiv.org/abs/2308.00352>>。

胡, Z., 伊森, A., 賈思, A., 基普夫, T., 岳, Y., 羅斯, DA, 施密德, C. 和法蒂, A. (2024)。“Scenecraft: LLM 代理要將 3D 場景合成為 Blender 代碼，<<https://arxiv.org/abs/2403.01248>>。

Häußler, M.、Esser, S. 和 Borrmann, A. (2021 年)。“通過集成 BIM 對鐵路設計進行規範合規性檢查，BPMN 和 DMN。建築自動化, 121, 103427.

國際標準化組織 (2024)。“ISO 16739-1：用於施工中資料共享的行業基礎類 (IFC) 2024：和設施管理行業 - 第 1 部分：數據架構，<<https://www.iso.org/standard/84123.html>>。
<https://www.iso.org/standard/84123.html> (最後訪問時間：2024 年 8 月 8 日)。

Jang, S., Lee, G., Oh, J., Lee, J. 和 Koo, B. (2024)。“使用 NADIA 自動細化外牆細節：自然 - 通過與 AI 交互實現基於語言的建築細節。高級工程資訊學, 61, 102532.

劉易斯, P., 佩雷斯, E., 皮克圖斯, A., 佩特羅尼, F., Karpukhin, V., Goyal, N., Küttler, H., 路易斯, M., Yih, WT, Rocktäschel, T., Riedel, S. 和 Kiela, D. (2020 年)。“用於知識密集型 NLP 任務的檢索增強生成。”論文集第 34 屆神經資訊處理系統國際會議, NIPS '20, 美國紐約州雷德胡克，Curran Associates 公司

李, C., 張, T., 杜, X., 張, Y. 和謝, H. (2024a)。“用於建築設計的生成式 AI：文獻綜述，<<https://arxiv.org/abs/2404.01335>>。

李, P., 李, B. 和李, Z. (2024b)。“草圖到架構：產生式 AI 輔助建築設計 / <<https://arxiv.org/abs/2403.20186>>。

廖偉、盧曉、飛、顧、Y. 和黃 Y. (2024 年)。“建築結構的生成式 AI 設計。”自動化建築, 157, 105187。

Lin, C.-H., Gao, J., Tang, L., Takikawa, T., Zeng, X., Huang, X., Kreis, K., Fidler, S., Liu, M.-Y., 和 Lin, T.-Y. (2023)。“Magic3d：高解析度文本到 3D 內容創建。”IEEE 電腦視覺與模式會議

認可 (CVPR)。

羅, Z. 和黃, W. (2022 年)。“平面圖：向量住宅平面圖對抗生成。”*自動化建築*, 142, 104470.

Mehta, N., Teruel, M., 鄧, X., 菲格羅亞-桑斯, S., 阿瓦達拉, A., 和 基塞列娃, J. (2024)。“改進接地氣通過幫助反饋與座席互動，在協作環境中理解語言。發現計算語言學協會：EACL 2024, Y. Graham 和 M. Purver, 編輯, 馬爾他聖朱利安, 計算語言學協會, 1306-1321年, <<https://aclanthology.org/2024.findings-eacl.87>> (3 月)。

Mescheder, L., Oechsle, M., Niemeyer, M., Nowozin, S., 和 Geiger, A. (2019)。“佔用網絡：學習功能空間的 3D 重建。2019 IEEE/CVF 計算機視覺與模式識別學術會議 (CVPR), 4455-4465。

米爾登霍爾, B., 斯裡尼瓦桑, PP, 坦西克, M., 巴倫, JT, 拉馬莫蒂, R. 和吳, R. (2021 年)。“Nerf：代表場景作為視圖合成的神經輻射場。公社。ACM, 65 (1), 99-106。

米斯特拉爾 (2024a)。“函數調用, <https://docs.mistral.ai/capabilities/function_calling>。訪問時間：2024年7月16日。

米斯特拉爾, AT (2024b)。“西北風大 2, <<https://mistral.ai/news/mistral-large-2407/>>。訪問時間：2024 年 8 月 3 日。

Nuyts, E., Bonduel, M. 和 Verstraeten, R. (2024 年)。“自動化合規性方法的比較分析檢查施工數據。高級工程資訊學, 60, 102443.

Oleynikova, H., Millane, A., Taylor, Z., Galceran, E., Nieto, JI 和 Siegart, RY (2016)。“有符號距離欄位：<<https://api.semanticscholar.org/CorpusID:28083959>>, 這是地圖繪製和規劃的自然表示形式。

OpenAI (2024a)。“你好 gpt-4o, <<https://openai.com/index/hello-gpt-4o/>>。訪問時間：2024年7月16日。

OpenAI (2024b)。“Openai 函數調用指南, <<https://platform.openai.com/docs/guides/function-calling>>。交流-停產時間：2024 年 7 月 16 日。

公園, JS, 奧布萊恩, JC, 蔡, CJ, 莫裡斯, MR, 梁, P. 和伯恩斯坦, MS (2023 年)。“產生代理：人類行為的互動式類比, <<https://arxiv.org/abs/2304.03442>>。

Pauwels, P., Deursen, DV, Verstraeten, R., Roo, JD, Meyer, RD, de Walle, RV 和 Campenhout, JVM (2011)。“一個用於建築性能檢查的語義規則檢查環境。建築自動化, 20, 506-518。

普爾, B., 賈恩, A., 巴倫, JT 和米爾登霍爾, B. (2022 年)。“Dreamfusion：使用 2D 擴散將文本轉換為 3D。”ArXiv, abs/2209.14988.

Pouliou, P., Horvath, AS 和 Palamas, G. (2023 年)。“推測性混合體：研究概念通過使用 3D 生成對抗網路來構建建築形式。國際建築學雜誌計算, 21 (2), 315-336。

Preidel, C. 和 Borrmann, A. (2018)。“基於 BIM 的代碼合規性檢查。”*建築資訊模型 / 施普林格國際出版社*, 367-381。

拉德福德, A., 金, JW, 徐, T., 布羅克曼, G., 麥克利維, C. 和薩茨克弗, I. (2022 年)。“強大的語音辨識”。

- 通過大規模的弱監管，<<https://arxiv.org/abs/2212.04356>>。
- Radford, A.、Metz, L. 和 Chintala, S. (2015 年)。“具有深度卷積的無監督表示學習生成對抗網路。CoRR, abs/1511.06434。
- Shabani, MA, Hosseini, S. 和 Furukawa, Y. (2023)。“房屋擴散：通過擴散生成向量平面圖具有離散和連續降噪的模型。2023 IEEE/CVF 計算機視覺與碼型會議認可 (CVPR), 5466–5475。
- Solihin, W. 和 Eastman, C. (2015)。“用於自動化 BIM 規則檢查開發的規則分類。”*自動化在建築中*, 53, 69-82。
- Stigsen, M., Moisi, A., Rasoulzadeh, S., Schinegger, K. 和 Rutzing, S. (2023)。“Ai diffusion 作為設計詞彙 - 研究 AI 圖像生成在早期建築設計和教育中的應用。587–596 (01).
- 孫, C., 韓, J., 鄧, W., 王, X., 秦, Z. 和古爾德, S. (2024)。“3D-GPT: 大型 3D 程式化建模語言模型、<<https://arxiv.org/abs/2310.12945>>。
- 孫 C.、周 Y. 和韓 Y. (2022)。“自動生成用於歷史城市改造的建築立面使用生成對抗網路。建築與環境, 212, 108781.
- Sydora, C. 和 Stroulia, E. (2020 年)。“基於規則的合規性檢查和建築內部的生成式設計 BIM 的。建築自動化, 120, 103368.
- Tomczak, A., v Berlo, L., Krijnen, T., Borrmann, A., 和 Bolpagni, M. (2022)。“對要指定數位建築項目中的資訊要求。IOP 會議系列：地球與環境科學, 1101, 092024.
- Tono, A. 和 Fischer, M. (2022)。“維特魯維奧：通過單透視圖草圖實現 3D 建築網絡，<<https://arxiv.org/abs/2210.13634>>。
- Vectorworks, D. (2024 年)。“Sdk 示例，<<https://github.com/VectorworksDeveloper/SDKExamples>>。存取：2024-07-16.
- 王 L., 馬 C., 馮 X., 張 Z., 楊 H., 張 J., 陳 Z., 唐 J., 陳 X., 林 Y., 趙 W. X., 魏 Z., 馮 X., 張 Z., 楊 H., 張 J., 陳 Z., 唐 J., 陳 X., 林 Y., 趙 W. X., 魏 Z., 馮 X., 張 Z., 楊 H., 張 J., 陳 Z., 唐 J., 陳 X., 林 Y., 趙 W. X., 魏 Z., 馮 X., 張 Z., 楊 H., 張 J., 陳 Z. 和溫, J. (2024 年)。“對基於大型語言模型的自主代理的調查。”*計算機科學前沿*, 18(6).
- Wang, S., Zeng, W., Chen, X., Ye, Y., Qiao, Y., 和 Fu, C.-W. (2021)。“Actfloor-gan: 活動引導的對抗性網路進行以人為本的平面圖設計。IEEE 視覺化與電腦圖形學匯刊, PP, 1–1.
- 魏, J., 博斯馬, M., 趙, VY, 顧, K., 於, AW, 萊斯特, B., 杜, N., 戴, AM 和樂, QV (2022 年)。“微調<<https://arxiv.org/abs/2109.01652>>, 語言模型是零樣本學習者。
- 魏, J., 王, X., 舒爾曼斯, D., 博斯馬, M., 伊希特, B., 夏, F., 池, EH, 樂, QV 和周, D. (2024)。“思維鏈提示在大型語言模型中引發推理。”第 36 屆國際會議記錄

神經資訊處理系統會議，NIPS '22，美國紐約州雷德胡克，Curran Associates Inc.

Wu, J., Nousias, S. 和 Borrmann, A. (2023)。“用於模型修復的基於參數化的解決方案空間探索。”

第 30 屆工程智慧計算國際會議 (EG-ICE) 論文集 (7 月)。

徐, X., 王, Y., 徐, C., 丁, Z., 江, J., 丁, Z. 和 卡爾森, BF (2024)。“關於遊戲代理的調查
和大型模型：方法、應用和挑戰，<<https://arxiv.org/abs/2403.10249>>。”

Yang, X., Wu, Y., Zhang, K., 和 Jin, C. (2021)。“Cpcgan：一種可控的 3D 點雲生成對抗網路
with semantic label generating。AAAI 人工智慧會議論文集，35 (4)，3154–3162。

Zhang, J. 和 El-Gohary, NM (2017)。“將語義 NLP 和邏輯推理集成到一個統一的系統中
全自動代碼檢查。建築自動化，73，45-57.

張, L., 鄭, L., 陳, Y., 黃, L. 和 周, S. (2022 年)。“Cgan 協助翻新風格和功能
街道立面——中國福建武夷地區的案例研究。可持續性，14,16575。

Zheng, J. 和 Fischer, M. (2023)。“用於 BIM 資訊搜索的基於動態提示的虛擬助手框架。”
建築自動化，155，105067.

周, Y.C., 鄭, Z., 林, JR 和 盧, XZ (2022 年)。“為複雜規則集成 NLP 和上下文無關語法
解釋為自動合規性檢查。工業計算機，142。

莊, X., Ju, Y., Yang, A. 和 Caldas, L. (2023)。“3D 建築體積的綜合和生成
生成建模。國際建築計算雜誌，21 (2)，297-314。

Çelen, A., Han, G., Schindler, K., Gool, LV, Armeni, I., Obukhov, A., 和 Wang, X. (2024)。“I-Design：個性化
<<https://arxiv.org/abs/2404.02838>> 室內設計師法學碩士。